



CRM-Tec

REPORTE TÉCNICO

PARA OBTENER EL TÍTULO DE

**TÉCNICO SUPERIOR UNIVERSITARIO EN
DESARROLLO DE SOFTWARE MULTIPLATAFORMA**

PRESENTA

CRISTIAN ALEJANDRO GONZALEZ CAMACHO

EDUARDO MONTOYA DUEÑAS

IRVIN DAVID REYES ROMANO

ALEXIS SANCHEZ AQUINO

IAN DANIEL VERA HERNÁNDEZ

ASESOR ACADÉMICO: ING. HUGO HUMBERTO ARVIZU ROSAS

ENCARGADO DE LA ACADEMIA: MTRO. FREDDY MANRIQUE FRANCO

ORGANIZACIÓN: "UNIVERSIDAD TECNOLÓGICA DE TECÁMAC"

CUATRIMESTRE: ENERO – ABRIL 2026

ÍNDICE

RESUMEN.....	1
ABSTRACT.....	3
OBJETIVOS.....	4
PLAN DE TRABAJO.....	8
CRONOGRAMA DE ACTIVIDADES.....	9
MARCO TEÓRICO.....	10
METODOLOGÍA.....	18
CAPÍTULO 1. ANÁLISIS.....	21
1.1 Marco legal.....	21
1.2 Recopilación y análisis de información.....	25
1.3 Problemática.....	26
1.4 Propuesta de solución.....	27
1.5 Obtención de requerimientos funcionales y no funcionales.....	28
1.5.1 Plantillas SRC Para la definición de requisitos (Funcionales).....	31
CAPÍTULO 2. DISEÑO DE INTERFACES GRÁFICAS DE USUARIO.....	48
2.1 Desarrollo de la Base de datos.....	48
2.1.1 Diseño.....	48
2.2 Desarrollo de interfaces.....	65
CAPÍTULO 3. DISEÑO E IMPLEMENTACIÓN DE LOS MÓDULOS.....	72
CONCLUSIONES.....	87
LISTADO DE SIGLAS O ACRÓNIMOS.....	88
GLOSARIO DE TÉRMINOS.....	89
REFERENCIAS.....	91

RESUMEN

En M-Tec, somos un equipo de desarrollo enfocado en democratizar el acceso a la tecnología para el sector empresarial. Nuestro objetivo principal es crear software de apoyo que sea altamente accesible, intuitivo y eficiente, diseñado específicamente para satisfacer las necesidades operativas de las pequeñas y medianas empresas (PyMEs). Creemos que la transformación digital no debe ser un privilegio exclusivo de las grandes corporaciones, por lo que trabajamos para brindar herramientas de código abierto que impulsen el crecimiento, la organización y la competitividad de los negocios locales.

En la actualidad, la gestión administrativa y operativa de las PyMEs representa un desafío crítico. Muchos emprendimientos dependen de métodos de control manuales para sus finanzas, inventarios y registro de clientes, lo que genera procesos tediosos y propensos a errores humanos. Esta falta de centralización de datos provoca una comunicación deficiente, el olvido de pedidos y una gestión ineficaz que impide alcanzar los objetivos de negocio. Aunado a esto, existe una brecha tecnológica significativa y una fuerte vulnerabilidad ante los constantes cambios en las normativas fiscales y la protección de datos sensibles, lo que pone en riesgo legal y financiero a estas unidades económicas.

Para hacer frente a estos retos, M-Tec ofrece y desarrolla CRM-Tec, una aplicación de escritorio integral orientada a la gestión empresarial. Nuestra solución busca centralizar las operaciones vitales del negocio mediante un entorno gráfico dinámico e intuitivo. La plataforma incorpora herramientas de gestión automatizada de stock, control de contabilidad y un innovador asistente virtual impulsado por inteligencia artificial para brindar soporte financiero interactivo.

Para lograr el desarrollo e implementación de este proyecto, hace uso del siguiente conjunto de herramientas, lenguajes y servicios tecnológicos:

Hardware

- Equipos de cómputo con 8GB RAM y procesadores Intel-Core y Ryzen.

Software

- Sistema Operativo *Windows 10 y Windows 11*
- Java
- JavaFX
- Scene Builder:
- Supabase (PostgreSQL)
- Visual Studio Code:
- StartUML modelador de diagramas UML

Servicios

- Internet Infinitum.
- Servicio de datos móviles por número telefónico

Como resultado de este proceso de ingeniería de software, se obtiene una plataforma CRM completamente funcional, escalable y adaptada al contexto normativo mexicano. El sistema logra consolidar módulos operativos clave, permitiendo una gestión de contactos eficiente, el seguimiento de tareas y agenda, el control de inventarios y la visualización gráfica de balances financieros.

ABSTRACT

Currently, Small and Medium-sized Enterprises (SMEs) face critical challenges in their operational management due to a reliance on manual processes, the decentralization of information, and vulnerability to tax regulations. To mitigate these deficiencies, this project details the development of CRM-Tec, a desktop application aimed at democratizing access to open-source administrative tools.

With a strong focus on multi-platform software development, the main objective is to centralize accounting, inventory control, and customer management within a dynamic and intuitive graphical environment.

The solution was built using Java and JavaFX, relying on a Cloud-Native architecture with Supabase (PostgreSQL) to ensure data scalability, integrity, and availability. Additionally, the system integrates an artificial intelligence-powered virtual assistant that provides interactive financial support in real time.

As a result of this technical effort, a functional system adapted to the Mexican business context was achieved, which automates task tracking and consolidates key operational modules to boost the competitiveness and organization of local businesses.

OBJETIVOS

Objetivo general

Implementar una solución tecnológica de escritorio orientada a la gestión empresarial integral, aplicando las metodologías de desarrollo que permitan a los usuarios migrar hacia la nube, con el propósito de centralizar los procesos de contabilidad, inventario y atención al cliente en una plataforma única, segura y eficiente que optimice la operatividad de las PyMEs.

Objetivos específicos

- Desarrollar una aplicación de escritorio bajo el modelo Open Source utilizando el lenguaje de programación Java, que permita agilizar las tareas contables y la administración de recursos de pequeños y medianos negocios mediante una arquitectura escalable.
- Implementar un asistente virtual con capacidades de análisis lógico y realista, con la finalidad de optimizar la consulta de información en tiempo real y proporcionar soporte financiero interactivo al usuario basado en normativas vigentes.
- Diseñar una interfaz gráfica intuitiva y dinámica centrada en la visualización de datos, que transforme registros contables y de inventario complejos en reportes estadísticos gráficos fáciles de interpretar para la toma de decisiones estratégicas.
- Garantizar la integridad y disponibilidad de la información mediante una arquitectura "Cloud-Native", permitiendo el flujo de operaciones administrativas de manera continua, incluso ante fallas del hardware del usuario, la información queda respaldada.

**UNIVERSIDAD TECNOLÓGICA
DE TECÁMAC
DIVISIÓN TIC
PROGRAMA DE ESTADÍAS
PROFESIONALES**



PROGRAMA DE TRABAJO

FECHA: 07/01/2026

DATOS DEL ALUMNO

NOMBRE:	Vera Hernández Ian Daniel
DIVISIÓN:	Tecnologías de la Información y Comunicación
CARRERA:	Técnico Superior Universitario en Tecnologías de la Información Área Desarrollo de Software Multiplataforma
MATRÍCULA:	2525160000
GENERACIÓN:	Enero – abril 2026

ASESOR UNIVERSITARIO

NOMBRE:	Arvizu Rosas Hugo Humberto
CARGO:	Profesor de Asignatura

PERÍODO

DURACIÓN:	15 Semanas
FECHA DE INICIO:	7 de Febrero de 2026
FECHA DE TERMINACIÓN:	15 de Abril de 2026

PROYECTO

NOMBRE:	CRM-Tec
DESCRIPCIÓN:	Aplicación de gestión para emprendimientos pequeños y medianos
OBJETIVO GENERAL:	Diseñar e implementar una solución tecnológica de escritorio para la gestión empresarial, aplicando metodologías de desarrollo aprendidas en la UTTEC, con el fin de centralizar procesos de contabilidad, inventario y atención al cliente en una plataforma única y eficiente.
OBJETIVOS ESPECÍFICOS:	-Desarrollar una aplicación de escritorio Open Source que se adapte a las necesidades específicas de los clientes y gracias a ello agilice tareas contables de pequeños y medianos negocios. -Implementar un asistente virtual con reconocimiento de voz e inteligencia artificial para agilizar la consulta de datos y proporcionar soporte financiero interactivo al usuario. - Diseñar una interfaz gráfica intuitiva centrada en la visualización de datos, que transforme registros contables complejos en estadísticos gráficos fáciles de interpretar.
ALCANCE(S):	Buscamos implementar un software de código abierto que facilite la gestión de negocios y empresas para presentar en el periodo Enero-Abril como parte de nuestro programa educativo y como proyecto integrador
META(S):	- Una interfaz que sea intuitiva y llamativa para todos los usuarios - Funcionalidad que concuerde con los datos requeridos por el cliente y asegure una guía financiera, que si bien, no infalible, pero si bastante confiable
RECURSOS:	Hardware Laptops (Ryzen 5 7530U, Ryzen 5 3450U, Ryzen 5 , Corei7

	16GB) PC escritorio (Ryzen 7 8700g) -Internet Software - Visual studio code - GitHub - PostgreSQL - Supabase
--	---

PLAN DE TRABAJO

ACTIVIDAD		DESCRIPCIÓN	SEMANA		FECHAS	
			INICIO	TÉRMINO	INICIO	TÉRMINO
1		Análisis del caso	1	3	07/01/2027	23/01/2027
	1.1	Recopilación y análisis de información				
	1.2	Determinación de problemática				
	1.3	Propuestas de solución				
	1.4	Determinación de requerimientos Funcionales				
	1.5	Creación de diagrama de Caso de uso general del sistema				
2		Desarrollo y diseño	4	7	26/01/2027	20/02/2027
	2.1	Diseño de Base de datos				
	2.2	Creación de Mockups				
	2.3	Diseño de Interfaces y formularios				
	2.4	Codificación del sistema				
3		Implementación y pruebas	8	15	23/02/2027	17/04/2027
	3.1	Pruebas de conexión con base de datos				
	3.2	Pruebas de funcionalidad del sistema				
	3.3	Retroalimentación y solución de errores				

CRONOGRAMA DE ACTIVIDADES

#	Actividades	Control	Enero				Febrero				Marzo				Abril		
			1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	Análisis	PROG.															
		REAL.															
2	Diseño y desarrollo	PROG.															
		REAL.															
3	Implementación y pruebas	PROG.															
		REAL.															

MARCO TEÓRICO

Para la planeación y realización de especificaciones, diseños, módulos, y apartados en general, fue necesario el uso de varias herramientas y aplicaciones de software, tanto para el diseño como codificación del sistema/aplicación. En el presente apartado, buscaremos presentar el uso de todas estas herramientas, en las cuales presentaremos el que son, al igual que ciertas características, como las ventajas y desventajas que estas nos pueden llegar a ofrecer para la realización del proyecto.

Windows 11

Windows 11 es la versión más reciente del sistema operativo de Microsoft, lanzado oficialmente el 5 de octubre de 2021, como una actualización gratuita para equipos con Windows 10, construida sobre el núcleo de Windows 10 pero optimizada para la productividad moderna. Windows 11 actúa como la plataforma a utilizarse donde los recursos de hardware como los IDEs y los servidores de bases de datos funcionen de manera estable.

Windows 11 es la base donde conviven los elementos para el funcionamiento del proyecto en general, ofreciendo herramientas nativas que facilitan la colaboración entre los variados roles que desenvolverá cada integrante del proyecto.

Algunas características que posee este sistema operativo son:

- Permite ejecutar un entorno Linux real directamente en Windows 11.
- Integración de Terminal Windows, una consola moderna que soporta múltiples pestañas, ideal para monitorear los logs de una IA y consultas de la base de datos simultáneamente.
- Protege el código fuente y las credenciales de acceso a los servidores de desarrollo contra malware.

Ventajas:

- **Compatibilidad Universal:** Al ser el sistema operativo más utilizado en entornos corporativos, garantiza que los drivers de bases de datos y las máquinas virtuales de Java funcionen sin conflictos de compatibilidad.
- **Soporte Nativo para IA:** Windows 11 incluye de base la optimización para modelos de IA y aprendizaje automático.
- **Integración con Ecosistemas de Desarrollo:** Herramientas como Visual Studio Code y GitHub Desktop están optimizadas para Windows 11.

Desventajas

- **Altos Requisitos de Hardware:** Windows 11 exige procesadores modernos y módulos de RAM amplios. Esto puede excluir a miembros del equipo con equipos antiguos.
- **Telemetría y Procesos en Segundo Plano:** El sistema operativo ejecuta muchos procesos internos de actualización y recolección de datos que pueden consumir recursos que el IDE o el servidor local de base de datos necesitan.
- **Actualizaciones Forzadas:** En momentos críticos de desarrollo o presentaciones, las actualizaciones automáticas de Windows pueden interrumpir el flujo de trabajo si no se gestionan correctamente.

Visual Studio Code

Visual Studio Code es un editor de código fuente ligero, pero extremadamente potente desarrollado por Microsoft. A diferencia de un IDE pesado como Visual Studio convencional, VS Code está diseñado para ser modular y altamente personalizable. Es multiplataforma (Windows, macOS, Linux) y se ha convertido en el estándar de la industria para proyectos de código abierto debido a su arquitectura basada en extensiones.

Visual Studio Code actúa como el "centro de mando" donde se integra el código del backend (lógica de asignación de tareas), el frontend (interfaz sencilla para el usuario) y la comunicación con los modelos de Inteligencia Artificial.

Ventajas:

- Bajo Consumo de Recursos: VS Code es rápido y ligero. Permitiendo que el desarrollo del CRM sea fluido incluso en equipos con especificaciones técnicas modestas.
- Personalización del Entorno: Puedes configurar "Workspaces" específicos para el CRM, donde las herramientas de manejo de inventario y gráficas estén a un clic de distancia.
- Soporte Multilenguaje: Visual Studio Code permite trabajar con múltiples lenguajes de manera simultánea bajo una misma interfaz.
- Depuración Avanzada: Permite pausar la ejecución del código para inspeccionar por qué una tarea no se asignó correctamente.

Desventajas:

- Configuración Manual Inicial: Al ser un editor modular, VS Code no viene "listo para usar" con todas las herramientas de IA o bases de datos. Requiere una inversión de tiempo buscando e instalando las extensiones correctas para que el CRM funcione.
- Dependencia de Extensiones de Terceros: Algunas funciones críticas del proyecto pueden depender de extensiones que no siempre están actualizadas, lo que podría generar conflictos en el futuro.

Java

Java es un lenguaje de programación de alto nivel, orientado a objetos y multiplataforma, que se ejecuta sobre la Máquina Virtual de Java (JVM). Su filosofía ha sido "Escribe una vez, ejecútalo en cualquier lugar" (*Write Once, Run Anywhere*), lo que significa que el código compilado puede ejecutarse en cualquier dispositivo que tenga instalada la JVM.

Java es el lenguaje ideal para el desarrollo del proyecto, por su robustez permite manejar múltiples conexiones simultáneas de usuarios, procesar grandes volúmenes de datos de inventario y servir como puente para integrar modelos de inteligencia artificial mediante bibliotecas especializadas o APIs.

Características Principales:

- **Orientación a Objetos (POO):** Java utiliza clases y objetos para estructurar el software. Esto facilita enormemente la creación de la agenda de contactos, permitiendo definir una clase base y derivar de ella "Cliente", "Proveedor" o "Empleado"
- **Gestión Automática de Memoria:** El sistema se encarga de liberar la memoria que ya no se utiliza. Crítico para un CRM que maneja gráficas y reportes pesados, evitando que la aplicación se vuelva lenta o se cierre por falta de recursos.
- **Multihilo:** Java permite realizar varias tareas al mismo tiempo. Esto permite que mientras el usuario está consultando a la IA sobre una duda de inventario, el sistema pueda seguir procesando la asignación de tareas en segundo plano sin interrumpir la experiencia.
- **Seguridad Robusta:** Java fue diseñado con un fuerte enfoque en la seguridad, incluyendo un "sandbox" donde se ejecuta el código y una verificación estricta de tipos

Ventajas:

- **Ecosistema y Librerías:** Existe una cantidad inmensa de bibliotecas de código abierto para prácticamente cualquier necesidad
- **Escalabilidad Empresarial:** Java está diseñado para crecer. Si termina manejando miles de proveedores y millones de registros de inventario, Java tiene la arquitectura necesaria para soportar esa carga.
- **Comunidad Global:** Al ser un proyecto de código abierto, utilizar Java garantiza que siempre habrá documentación, foros y desarrolladores dispuestos a contribuir o solucionar errores.
- **Conectividad con Bases de Datos:** A través de JDBC (Java Database Connectivity), la integración de PostgreSQL es sumamente estable y eficiente para la gestión de inventarios.

Desventajas:

- Consumo de memoria: Java tiende a consumir más memoria RAM que otros lenguajes debido a la ejecución de la JVM. Esto podría ser una barrera si el CRM se pretende ejecutar en equipos muy antiguos.
- Verbosidad del Código: Para realizar tareas sencillas, Java requiere más líneas de código que lenguajes como Python o Kotlin. Esto puede hacer que el desarrollo inicial sea un poco más lento.

JavaFX

JavaFX es una extensión de software (propia de Java) para crear y entregar aplicaciones de escritorio, así como aplicaciones ricas de internet (RIAs) que pueden ejecutarse en una amplia variedad de dispositivos. Es la evolución moderna de la antigua biblioteca Swing, diseñada para proporcionar una interfaz de usuario (UI) visualmente atractiva y altamente personalizable.

JavaFX es la "cara" del proyecto; es la herramienta que permite construir las ventanas, botones, tablas y formularios con los que va a interactuar el usuario final. Gracias a esta tecnología, el CRM deja de ser solo código y se convierte en una experiencia visual intuitiva para la gestión del negocio.

Este software se llegó a seleccionar por dos características muy importantes para el desarrollo de la aplicación:

- Separa la lógica y el diseño mediante archivos FXML.
- Capacidad de estilizado mediante CSS, similar al desarrollo web.

Ventajas:

- Diseño Moderno y Personalizable: Permite aplicar hojas de estilo CSS para que el CRM tenga una apariencia profesional y acorde a la identidad visual de la empresa, alejándose del aspecto gris y antiguo de las aplicaciones Java tradicionales.

- Integración con Scene Builder: Facilita el diseño de las interfaces de manera visual (arrastrar y soltar), acelerando el desarrollo de los módulos de captura de datos y visualización.
- Gráficos y Multimedia: Ofrece componentes nativos para generar reportes gráficos de ventas y estadísticas, vitales para la toma de decisiones en el CRM.

Desventajas:

- Configuración del Entorno: En las versiones modernas de Java (JDK 11 en adelante), JavaFX no viene incluido por defecto, lo que requiere una configuración adicional de módulos y librerías en el IDE.
- Consumo de memoria Gráfica: Las interfaces ricas y animadas pueden exigir más recursos de la tarjeta gráfica (GPU) de los equipos donde se instale el sistema.

Supabase

Supabase es una plataforma de "Backend como Servicio" (BaaS) de código abierto, que se posiciona como una alternativa directa a Google Firebase. Está construida sobre PostgreSQL, lo que le otorga la robustez de una base de datos relacional tradicional, combinada con herramientas modernas de desarrollo en la nube. Proporciona de manera automática una API completa, gestión de usuarios y almacenamiento de archivos sin necesidad de configurar servidores desde cero.

En el desarrollo del proyecto, Supabase actúa como la infraestructura en la nube que centraliza la lógica de datos. Funciona como el cerebro remoto que sincroniza la información entre los distintos usuarios del sistema, gestionando no solo el almacenamiento de la información de clientes y ventas, sino también la seguridad del acceso mediante su sistema de autenticación integrado.

Ventajas:

- Robustez Relacional en la Nube (PostgreSQL): A diferencia de las soluciones basadas en modelos NoSQL, la infraestructura de Supabase está respaldada

por PostgreSQL. En un entorno nativo de la nube, esto asegura un alto nivel de integridad referencial y el cumplimiento estricto de las propiedades ACID. Esta arquitectura es fundamental para el CRM, ya que permite la ejecución de consultas complejas (como operaciones *JOIN*) necesarias para cruzar información de ventas, clientes e inventario en la generación de reportes precisos.

- **Sincronización de Datos en Tiempo Real:** Al estar la aplicación conectada directamente a la infraestructura en la nube, se aprovecha la capacidad de suscribirse a eventos de la base de datos. Cualquier modificación operativa, como el registro de una nueva venta o una actualización en el stock, se transmite y refleja de manera instantánea en las pantallas de todos los usuarios activos. Esto optimiza el flujo de trabajo al eliminar la necesidad de que el usuario actualice la interfaz manualmente, garantizando que siempre se visualice la información más reciente.
- **Gestión de Identidades y Seguridad Integrada (RLS):** Supabase delega y centraliza la autenticación mediante un sistema robusto que incluye Seguridad a Nivel de Fila (*Row Level Security*). Para una aplicación de escritorio, esto simplifica significativamente la arquitectura de seguridad, ya que permite configurar reglas de acceso directamente en la base de datos en lugar de programar validaciones complejas en el código cliente. De esta forma, la creación y restricción de roles operativos (como "Administrador" o "Vendedor") se gestiona de manera nativa, asegurando que cada perfil interactúe únicamente con los datos autorizados.

Desventajas:

- **Dependencia Absoluta de Conectividad:** Al emplear una arquitectura Cloud-Native, la aplicación carece de almacenamiento local persistente para operar de manera autónoma, por lo que requiere una conexión a internet constante y estable. Si el usuario experimenta interrupciones en su red o se encuentra en una zona sin cobertura, perderá total o parcialmente el acceso al sistema, imposibilitando la consulta, creación o edición de registros (como clientes o ventas) hasta que se restablezca la conexión con el servidor.

- **Latencia y Límites de Peticiones en la Nube:** Dado que no existe una base de datos local que procese las solicitudes de manera inmediata, cada acción de lectura o escritura exige una petición a través de la red hacia los servidores de Supabase. Esto introduce tiempos de latencia que pueden afectar la fluidez de la interfaz gráfica si la conexión es lenta. Además, delegar toda la lógica y almacenamiento a la nube genera una alta dependencia de la infraestructura del proveedor, lo que implica monitorear constantemente el consumo de ancho de banda y el límite de peticiones a la API para evitar cuellos de botella o costos operativos imprevistos al escalar el sistema.

Scene Builder

Scene Builder es una herramienta de diseño visual para la plataforma JavaFX que permite la creación de interfaces de usuario mediante una metodología de "arrastrar y soltar" (drag-and-drop). Esta aplicación genera automáticamente código FXML, un lenguaje basado en XML que define la estructura de la interfaz gráfica separándola de la lógica de programación en Java.

Scene Builder funciona como el "taller de diseño" del proyecto. Es el entorno donde se construyen visualmente los formularios de registro, las tablas de inventario y los paneles de control del CRM. Permite a los desarrolladores ver exactamente cómo lucirá la aplicación final mientras la diseñan, sin necesidad de compilar y ejecutar el código constantemente para verificar cambios visuales.

Ventajas:

- **Separación de Responsabilidades (MVC):** Promueve una arquitectura limpia al mantener el diseño (FXML) separado del código lógico (Java), lo que facilita el mantenimiento y la escalabilidad del código fuente.
- **Prototipado Rápido:** Permite armar interfaces complejas en minutos simplemente arrastrando componentes como botones, tablas y etiquetas al lienzo de trabajo, acelerando drásticamente el tiempo de desarrollo.

- Integración con CSS: Facilita la aplicación de estilos visuales personalizados (colores, fuentes, bordes) mediante hojas de estilo CSS, permitiendo que el CRM tenga una apariencia moderna y profesional.

Desventajas:

- Generación de Código Verbo: En ocasiones, el código FXML generado automáticamente puede ser extenso y contener propiedades innecesarias que aumentan el tamaño de los archivos si no se limpian manualmente.
- Compatibilidad de Versiones: Es necesario asegurar que la versión de Scene Builder coincida con la versión del JDK (Java Development Kit) utilizado en el proyecto, ya que las discrepancias pueden causar que ciertos componentes visuales no se muestren o funcionen incorrectamente.

METODOLOGÍA

La metodología iterativa e incremental se utilizará para el desarrollo de la aplicación de escritorio que se describe a continuación, así como los ciclos / iteraciones del software.

El modelo se basa en la implementación de cambios consecutivos sin perder el camino del proyecto, esto se hace mediante ciclos repetitivos llamados iteraciones. Esta metodología es adecuada para el proyecto ya que los requerimientos no están completamente definidos dado el tipo de usuarios a los que apuntamos y nos da la oportunidad de realizar cambios, adaptar estos cambios al proceso actual y realizar actualizaciones sin perder el avance del proyecto. Esta metodología divide el sistema en incrementos funcionales donde cada incremento o fase va a agregar un valor al software y mediante pruebas se evaluará y se decidirá si se avanzará a la siguiente fase. El sistema mejorará de forma continua y esto reducirá el riesgo de cometer errores.

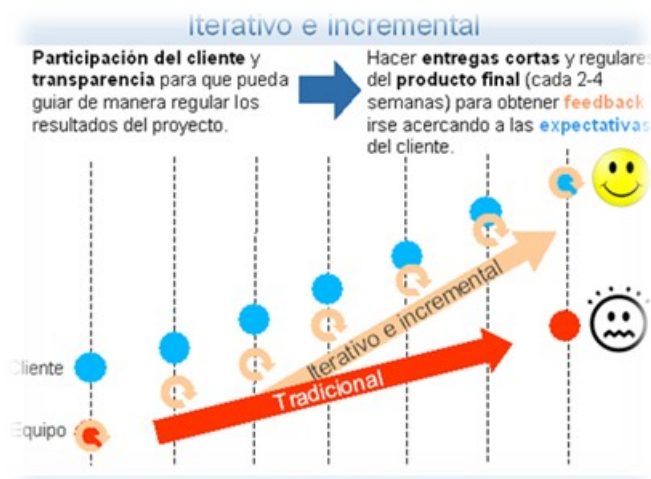


Figura 1 Grafica del modelo iterativo e incremental

Esta metodología se aplica de la siguiente manera al Proyecto de CRM-Tec:

1.- Definición de idea y alcance

Se establece la idea principal, el propósito, los objetivos (general y específicos) y el alcance que tiene el proyecto. Como el sistema no tiene requerimientos definidos, se plantea la visión general del proyecto con lo cual se identifican en el proceso las funciones y las limitaciones técnicas que tendrá este mismo.

2.- Análisis iterativo

Se realiza el análisis de manera incremental en cada ciclo/iteración, se identifican los requerimientos y se refinan conforme avanza el desarrollo del sistema. Cada requerimiento debe ser definido acorde a las necesidades del usuario y los resultados obtenidos de la iteración anterior.

3.- Diseño incremental

Se desarrolla cada componente de manera individual, por partes, comenzando por los componentes básicos como la arquitectura del proyecto, en este caso un CRM. Cada iteración puede variar dependiendo al tipo de requerimiento que se va a implementar, ya sea un requerimiento de UI (Interfaz de Usuario) o UX (Experiencia del Usuario).

4.-Desarrollo o implementación

Se implementan las funciones ya definidas en cada incremento, mediante esta fase se realizará la codificación de cada función y se podrá realizar su respectiva prueba para avanzar a la siguiente iteración.

5.-Pruebas continuas

Las pruebas se realizan al final de cada iteración para saber si la función cumple con lo establecido, así pasando a la siguiente iteración propuesta comprobando que su función es correcta.

6.-Retroalimentación

Al final de cada iteración se realiza una retroalimentación, esta se recopila creando un archivo .txt (README de GitHub) en el que se puede utilizar para mejorar la misma función o simplemente intercambiarla por otra más eficaz.

7.-Mantenimiento y mejora

Como anteriormente se dijo, se puede utilizar la retroalimentación para mejorar la función, para actualizarla o para cambiarla por algo más eficaz y eficiente. Otra opción

que se puede tomar en cuenta es que se pueda dar mantenimiento por si en algún momento existe un error.

CAPÍTULO 1. ANÁLISIS

1.1 Marco legal

Estandarización en el Software Administrativo:

La estandarización de los CRM ya no es una opción de calidad, sino una necesidad de cumplimiento. La evolución de la fiscalización electrónica del SAT exige que el software de ventas no solo capture datos, sino que los valide conforme a las reglas tributarias antes de generar documentos fiscales. Paralelamente, la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP) impone severas multas por el manejo inadecuado de información sensible del cliente.

Extensión: El CRM debe integrarse al ecosistema de la PyME como un punto de control de riesgo. La adopción del paradigma *Cloud-Native* garantiza la centralización, integridad y disponibilidad inmediata de la información fiscal y administrativa. Al depender de una infraestructura en la nube, el sistema asegura que todas las transacciones operativas y comerciales se registren y validen en tiempo real en una única fuente de verdad. Esto facilita el cumplimiento estricto de las normativas fiscales vigentes, permitiendo una auditoría concurrente sin el riesgo de discrepancias o pérdida de datos por sincronizaciones diferidas.

Marco Normativo y Fiscal: La Columna Vertebral del Desarrollo:

El diseño debe seguir el "Compliance by Design", incorporando las reglas fiscales como lógica de negocio.

Extensión Detallada:

- CFDI 4.0 (Anexo 20): La validación de identidad fiscal va más allá del RFC. El

CRM debe poder verificar la tripleta Nombre/Razón Social, Código Postal y Régimen Fiscal del receptor con las listas del SAT, evitando la emisión de facturas que serán rechazadas. El Patrón de Actualización de catálogos debe ser automático y transparente para el usuario, manteniendo actualizados elementos como claves de productos, unidades de medida y regímenes fiscales.

- Contabilidad Electrónica 1.3 (Anexo 24): El estándar exige que el CRM esté diseñado para la Trazabilidad Transaccional. Esto implica que cada ingreso o egreso registrado en el CRM debe tener el UUID asociado para vincularlo a las Pólizas Contables. El sistema debe usar internamente el Código Agrupador del SAT para facilitar la exportación de información contable al contador o ERP contable.
- Complementos Fiscales (Carta Porte y REP): La Carta Porte 3.0/3.1 es crítica para empresas que manejan logística. El CRM debe modelar el ciclo de vida del traslado, incluyendo la validación de direcciones con coordenadas geográficas y la correcta clasificación de las Mercancías. El Complemento de Pagos 2.0 (REP) es esencial para el flujo de efectivo: el sistema debe calcular automáticamente la proporción de impuestos trasladados (IVA, IEPS) correspondiente al pago parcial recibido, cumpliendo estrictamente con el anexo 20.

Arquitectura Técnica: Escalabilidad y el Paradigma "Cloud-Native":

Fundamentos Cloud-Native: Se estandariza el uso de PostgreSQL alojado en la nube (a través de la plataforma Supabase) por su probada robustez relacional y su soporte transaccional estricto (ACID). Este modelo centralizado elimina la complejidad de mantener bases de datos locales y resolver conflictos de sincronización. En su lugar, implementa un modelo de datos optimizado para la alta disponibilidad, donde las operaciones se reflejan instantáneamente en todos los clientes conectados, centralizando además las reglas de acceso mediante Políticas de Seguridad a Nivel de Fila (RLS).

Seguridad de la Información y Privacidad: Blindaje Legal y Técnico:

El software debe ser un custodio seguro de los datos personales y fiscales.

Extensión:

- Protección de Datos en Reposo: El cifrado de la base de datos con SQLCipher (AES-256) es obligatorio. La clave de cifrado no debe estar fija en el código, sino derivada de la contraseña única del usuario mediante funciones *Key Derivation Functions* (KDF), protegiendo la base de datos incluso si el disco duro es extraído.
- Protección de la Cadena de Suministro: La Firma Digital (Code Signing EV) verifica que el ejecutable no haya sido alterado por un tercero. La adopción de The Update Framework (TUF) es una defensa crítica contra la alteración de las actualizaciones (*supply chain attacks*), garantizando que las nuevas versiones provengan genuinamente del desarrollador.
- Cumplimiento LFPDPPP (Derechos ARCO): Para los Derechos de Cancelación, el sistema debe implementar un Estado de Bloqueo que impida el uso posterior de los datos del cliente sin eliminarlos inmediatamente (conservando un registro histórico no accesible, según la ley) y ofrecer la función de Borrado Seguro (sobrescritura binaria o *shredding*) para la eliminación permanente de la información que ya no deba ser conservada.

Ingeniería de Procesos y Calidad: Estándares para el Ciclo de Vida

La calidad del desarrollo debe ser medible y auditable.

Extensión:

- ISO/IEC 29110 y MoProSoft: Estos estándares son flexibles y escalables para equipos pequeños. Exigen la formalización de la gestión de requisitos (para evitar cambios constantes en el alcance) y la gestión de la configuración (para rastrear versiones y *bugs*). MoProSoft, en particular, ayuda a la PyME de desarrollo a estandarizar procesos que son reconocidos en México.
- Calidad del Producto (ISO/IEC 25010): Las métricas clave son:
 - Adecuación Funcional: Cobertura del 100% de los requisitos fiscales

aplicables.

- Usabilidad: Medición del tiempo que tarda un usuario promedio en completar tareas clave (ej. emitir una factura con REP) y su curva de aprendizaje.
- Eficiencia de Desempeño: Tiempos de respuesta inferiores a 1 segundo para la mayoría de las operaciones locales, incluso con bases de datos grandes, esencial para el hardware modesto de las PyMEs.

Interoperabilidad y Ecosistema Digital:

El CRM debe ser un componente integrado, no una isla de información.

Extensión:

- Integración Bancaria y CoDi (Banxico): La integración con CoDi permite generar un código QR de cobro directamente desde una cotización o factura. El sistema debe monitorear el *webhook* de Banxico para recibir la notificación de pago en tiempo real y realizar la Conciliación Automática contra la cuenta por cobrar.
- Integración con ERPs Legacy: Los sistemas como Aspel o CONTPAQi siguen siendo la columna vertebral contable de muchas PyMEs. El CRM debe exportar datos de ventas, cuentas por cobrar y clientes de manera segura, preferiblemente usando Archivos de Intercambio (Flat Files/Excel) con estructuras bien definidas, minimizando la necesidad de acceder directamente a las bases de datos de terceros.
- NOM-151: Este es el estándar para el valor legal de los documentos electrónicos. Al integrarse con un PSC, el CRM puede generar Constancias de Conservación para documentos críticos (contratos, cotizaciones aceptadas), que incluyen un Sello de Tiempo y la Firma Digital del PSC, otorgándoles la misma validez legal que un documento en papel notariado.

Mantenimiento, Soporte y Propiedad Intelectual

Extensión:

- Mantenimiento Adaptativo (ISO/IEC 14764): El diseño debe ser modular (basado en microservicios o patrones de diseño desacoplados) para que las

actualizaciones fiscales (ej. un cambio en la estructura del CFDI o el REP) solo requieran modificar un módulo específico del código, sin afectar la funcionalidad central del CRM.

- Documentación del Usuario (IEC 82079-1): La documentación debe ser centrada en tareas (cómo hago X, en lugar de qué hace X) e incluir advertencias de riesgos (ej. advertir al usuario sobre las consecuencias de omitir un dato fiscal o desactivar la sincronización).
- Protección de Derechos de Autor (INDAUTOR): El registro del código fuente (y su documentación asociada) ante el INDAUTOR establece una prueba legal de la fecha de creación y la autoría, esencial para proteger los activos de Propiedad Intelectual del desarrollador en México.

1.2 Recopilación y análisis de información

Para el desarrollo de la presente aplicación de escritorio de CRM (Customer Relationship Management), fue fundamental establecer un diagnóstico claro sobre las necesidades operativas de los pequeños y medianos negocios. El proceso de obtención de información se llevó a cabo mediante sesiones de lluvia de ideas (brainstorming) entre los integrantes del equipo, tomando como base el caso de estudio y la experiencia directa de Irvin David Reyes Romano en la gestión de su propio negocio.

En las sesiones de trabajo, se identificó que muchos pequeños negocios operan bajo esquemas tradicionales o manuales, lo que genera una fragmentación de la información de los clientes. Irvin David planteó la problemática recurrente de no contar con un repositorio centralizado, lo que dificulta el seguimiento de prospectos y la fidelización de clientes actuales.

A través del análisis de esta idea base, se determinaron los siguientes puntos críticos que el sistema debe resolver:

- Centralización de Datos: Actualmente, la información de contacto se encuentra dispersa en agendas físicas, chats de mensajería o archivos de texto aislados, lo que incrementa el riesgo de pérdida de datos.

- **Gestión de Interacciones:** Se detectó la falta de un histórico de comunicación con el cliente, lo que impide conocer qué productos o servicios se le han ofrecido previamente.
- **Optimización de Búsqueda:** El análisis reveló que el tiempo invertido en localizar a un contacto específico es excesivo cuando la cartera de clientes crece, evidenciando la necesidad de filtros avanzados (orden alfabético, fecha de agregación, etc.).
- **Priorización de Clientes:** Se identificó la necesidad de "fijar" o destacar contactos estratégicos para un acceso rápido, una funcionalidad crítica para la operatividad diaria de una PyME.

Además de las sesiones internas, se realizó una investigación sobre las herramientas CRM existentes en el mercado. Se concluyó que muchas soluciones actuales son demasiado complejas o costosas para un pequeño negocio, lo que validó la propuesta de desarrollar una herramienta de escritorio intuitiva, con integración de servicios conocidos como Google y GitHub para el inicio de sesión, facilitando así la adopción tecnológica por parte del usuario.

Finalmente, el análisis de la información recopilada permitió estructurar los módulos principales del sistema, enfocándose en la simplicidad de uso y la eficiencia en la gestión de contactos, asegurando que la herramienta sea un apoyo real para el crecimiento administrativo del negocio de Irvin y, por extensión, de otras PyMEs con problemáticas similares.

1.3 Problemática

En la actualidad, la gestión operativa y administrativa de las pequeñas y medianas empresas (PyMEs) representa un desafío crítico que compromete su estabilidad y crecimiento. El control manual de variables fundamentales como las ganancias, las pérdidas, el inventariado detallado y el registro histórico de ventas (tanto mensuales como anuales) se convierte en un proceso sumamente tedioso y propenso a errores humanos, especialmente para aquellos emprendedores que se encuentran en las etapas iniciales de su negocio.

Esta situación deriva en una serie de deficiencias operativas recurrentes, entre las que destacan el olvido o la mala gestión de los pedidos de los clientes y una comunicación deficiente entre los equipos de trabajo. Al no contar con un flujo de información centralizado, las áreas administrativas y operativas presentan dificultades para coordinar esfuerzos, lo que impide alcanzar los objetivos de negocio establecidos y genera un estancamiento en la productividad.

Aunado a lo anterior, existe una brecha tecnológica significativa debido al nulo o limitado manejo de entornos digitales especializados para la gestión empresarial. La dependencia de métodos tradicionales o herramientas desarticuladas provoca un desfase en la actualización de datos en tiempo real, lo que se traduce directamente en la pérdida de clientes potenciales y actuales, el deterioro de la imagen profesional de la empresa ante el mercado y, en última instancia, cuantiosas pérdidas económicas que ponen en riesgo la permanencia de la unidad económica.

La falta de una herramienta que integre las normativas fiscales vigentes y la protección de datos sensibles agrava el riesgo legal y financiero, dejando a las empresas vulnerables ante auditorías o incidentes de seguridad de la información.

1.4 Propuesta de solución

La propuesta de solución se sustenta en el análisis de la problemática identificada en las pequeñas y medianas empresas (PyMEs), las cuales enfrentan dificultades en la gestión de inventarios, procesos contables y comunicación interna debido a la falta de herramientas digitales integradas. Para ello, se propone el desarrollo de CRM-Tec, una aplicación de escritorio de código abierto diseñada para centralizar las operaciones vitales del negocio bajo un entorno gráfico intuitivo y eficaz que facilite la interacción entre el negocio, sus clientes y proveedores.

La implementación técnica se basa en el lenguaje de programación Java, aprovechando su robustez y portabilidad para asegurar que el sistema funcione de manera estable en diversos sistemas operativos como Windows, macOS y Linux. Bajo un paradigma de arquitectura *Cloud-Native*, la solución centraliza la gestión operativa

de la empresa aprovechando una infraestructura backend escalable. Esto permite el acceso y la actualización en tiempo real de la información, garantizando que todos los usuarios (como administradores o vendedores) interactúen simultáneamente con los datos más recientes, sin importar su ubicación. Al mantener una conexión directa con la base de datos en la nube, el sistema elimina las colisiones de información y asegura una trazabilidad inmediata en el control de inventarios, registro de clientes y emisión de ventas.

Como características funcionales principales, el sistema permitirá la gestión automatizada de stock con recordatorios de reabastecimiento y fechas de caducidad, el control de contabilidad para el seguimiento de ingresos y egresos, y la integración de una asistencia por IA para brindar consejos financieros basados en normativas de Condusef. Al ser un software de código abierto y modular, la propuesta ofrece una base escalable que permite a los desarrolladores externos adaptar y mejorar las funciones según las necesidades específicas de cada modelo de negocio. ponte a chambear mejor

1.5 Obtención de requerimientos funcionales y no funcionales

Para la obtención de los requerimientos funcionales y no funcionales, se llevó a cabo un proceso de análisis basado en la metodología iterativa e incremental adoptada por el equipo de trabajo. En una primera instancia, se realizó la definición del alcance y propósito del proyecto CRM-Tec , donde los integrantes presentaron sus perspectivas sobre las necesidades operativas de las PyMEs para determinar las funciones que el sistema debe integrar.

Los requisitos funcionales del sistema representan las acciones y comportamientos específicos que el software debe ejecutar para satisfacer las demandas del usuario, tales como la gestión de inventarios, el registro de contactos y la interacción con modelos de inteligencia artificial. Por su parte, los requisitos no funcionales especifican los criterios de calidad y operación, enfocándose en aspectos críticos como el rendimiento de la interfaz, la seguridad mediante algoritmos de hash y la portabilidad multiplataforma garantizada por el uso de Java.

Este levantamiento de requerimientos se consolidó a través de la fase de análisis iterativo, donde cada necesidad fue refinada conforme avanzó el entendimiento de la problemática comercial y los marcos legales vigentes, como la normativa de CFDI 4.0 y la LFPDPPP. De esta manera, se garantiza que los elementos contenidos en la aplicación no solo cumplan con los objetivos técnicos, sino que también brinden una solución robusta y eficiente para el personal administrativo y los dueños de negocios que operarán el sistema.

A continuación, en la figura 1.1, se muestra el diagrama general de casos de uso.



Figura 1.1 Diagrama general

1.5.1 Plantillas SRC Para la definición de requisitos (Funcionales)

ID Requisito	RF1
Nombre	Inicio de sesión
Descripción	Muestra un formulario de inicio de sesión, en donde, el usuario podrá ingresar sus datos para iniciar sesión con su cuenta existente, o en su caso crear su propio usuario si este no existe
Autor	Vera Hernández Ian Daniel
Entradas	-Correo del usuario -Contraseña
Salidas	Mensaje de "Error datos incompletos" Mensaje de "Error datos incorrectos" Pantalla de creación de usuarios Pantalla principal de CRM-Tec
Restricciones	Los datos deben coincidir con los usuarios almacenados en la base de datos, en caso contrario, el usuario no podrá acceder a la aplicación o deberá crear un usuario nuevo
Actor	Usuario

Figura 1.2 Requerimiento funcional 1

ID Requisito	RF2
Nombre	Inicio de sesión con plataformas externas (Google/GitHub)
Descripción	En la parte inferior del inicio de sesión se podrá hallar los botones "Iniciar sesión con...", al pulsar el botón (Github y google respectivamente) correspondiente el usuario podrá acceder a la aplicación con los datos proporcionados por Google mediante OAuth
Autor	Sánchez Aquino Alexis
Entradas	Token de inicio de sesión
Salidas	Mensaje de "acceso permitido" Mensaje de "Acceso denegado"

	Pantalla principal de CRM-Tec
Restricciones	El token proporcionado por OAuth debe ser válido, si no el usuario será reenviado al login Aunque el usuario inicie sesión con OAuth, si no tiene datos registrados de su empresa será reenviado a llenar los datos de su empresa para poder continuar
Actor	Usuario

Figura 1.3 Requerimiento funcional 2

ID Requisito	RF3
Nombre	Creación de usuarios
Descripción	Muestra una interfaz al pulsar el link “Crear cuenta” en la cual el actor usuario podrá acceder al formulario para crear una nueva cuenta
Autor	Reyes Romano Irvin David
Entradas	Nombre completo del usuario (nombre, apellido paterno, apellido materno), correo, contraseña, Datos de su empresa/negocio(Nombre negocio/empresa, calle, colonia, C.P, Municipio, Estado)
Salidas	Mensaje de “Cuenta Registrada” Mensaje de “Usuario ya existente” Mensaje de “Error datos incorrectos” Mensaje de “Error datos incompletos” Pantalla Login de CRM-Tec
Restricciones	Los datos deben coincidir con los usuarios almacenados en la base de datos, si no el usuario no podrá acceder a la aplicación o deberá crear un usuario nuevo
Actor	Usuario

Figura 1.4 Requerimiento funcional 3

ID Requisito	RF4
--------------	-----

Nombre	Visualización de los contactos
Descripción	Muestra un widget dentro de la interfaz "Principal" ubicado en la esquina inferior derecha bajo el nombre "Contactos" en el cual, se muestra una tabla con los últimos 4 contactos agregados recientemente (En orden descendente empezando por el último agregado). Dicha tabla mostrará el Nombre. Número y Ocupación del contacto
Autor	Montoya Dueñas Eduardo
Entradas	Datos de los Contactos registrados en la base de datos Fechas de registro de los contactos
Salidas	Widget "Contactos" Widget "Contactos" vacío en caso de datos inexistentes
Restricciones	Debe haber datos en la base de datos Los datos mostrados (Si hay) deben coincidir con los de la base de datos registrados al ID del cliente
Actor	Sistema

Figura 1.5 Requerimiento funcional 4

ID Requisito	RF5
Nombre	Actualización de los widget

Descripción	El usuario debe ir a la interfaz correspondiente de cada widget para poder ver sus datos actualizados en tiempo real dentro de la interfaz principal
Autor	Gonzales Camacho Cristian Alejandro
Entradas	Datos de a base de datos
Salidas	Interfaz de registro de usuarios
Restricciones	Únicamente se activa la función cuando el usuario tiene registros realizados en la aplicación
Actor	Usuario

Figura 1.6 Requerimiento funcional 5

ID Requisito	RF6
Nombre	Visualización del estado del inventario
Descripción	Muestra un widget dentro de la interfaz “Principal” ubicado en la esquina superior derecha bajo el nombre “Estado del inventario” en el cual se debe ver el estado del inventario en dos posibles estados/mensajes: “Abastecido” “Falta abastecer”
Autor	Montoya Dueñas Eduardo
Entradas	Número de productos en el inventario Fechas de registro de los clientes
Salidas	Widget “Estado del inventario”

	Widget “Estado del inventario” vacío en caso de datos inexistentes
Restricciones	Debe haber datos en la tabla stock El mensaje “Falta abastecer” solo aparecerá si se llega al número mínimo de artículos Los datos mostrados (Si hay) deben coincidir con los de la base de datos
Actor	Sistema

Figura 1.7 Requerimiento funcional 6

ID Requisito	RF7
Nombre	Visualización de gráficas de balance
Descripción	Muestra un widget dentro de la interfaz “Principal” ubicado en la parte derecha inferior bajo el nombre “Ingresos y egresos” El cual mediante la información en la base de datos mostrará una gráfica que muestre los ingresos y egresos del mes
Autor	Vera Hernández Ian Daniel
Entradas	Balance del negocio Registro de movimientos
Salidas	Widget “Gráfica” Los datos mostrados (Si hay) deben coincidir con los de la base de datos registrados al ID del usuario
Restricciones	Debe haber datos del balance en la base de datos

	El usuario debe llevar al menos una semana de datos registrados
Actor	Sistema

Figura 1.8 Requerimiento funcional 7

ID Requisito	RF8
Nombre	Visualización de los eventos programados
Descripción	Muestra un widget dentro de la interfaz “Principal” ubicado entre el widget de Inventario y el de contactos bajo el nombre “Agenda” el cual mostrará en una pequeña tabla la fecha actual y el evento próximo a la misma
Autor	Vera Hernández Ian Daniel
Entradas	Datos de la base de datos
Salidas	Widget “Agenda” Widget “Agenda” vacío en caso de datos inexistentes
Restricciones	Debe haber datos en la base de datos Los datos mostrados (Si hay) deben coincidir con los de la base de datos registrados al ID del cliente
Actor	Sistema

Figura 1.9 Requerimiento funcional 8

ID Requisito	RF9
Nombre	Buscar y filtrar contactos

Descripción	Mediante la barra de búsqueda superior en la lista de contactos se podrá consultar el contacto que coincida con el nombre escrito, el teléfono o en su caso el correo. Para filtrarlos por ocupación el usuario tendrá a la izquierda una serie de botones con las diferentes ocupaciones disponibles
Autor	Gonzales Camacho Cristian Alejandro
Entradas	Búsqueda a realizar
Salidas	Lista de contactos que coinciden con la información ingresada Mensaje de “Contacto no encontrado”
Restricciones	El nombre ingresado debe coincidir con el contacto registrado, si el contacto no está registrado se lanzará el mensaje de error.
Actor	Usuario

Figura 1.10 Requerimiento funcional 9

ID Requisito	RF10
Nombre	Registro de contacto
Descripción	Al presionar el botón debajo de la lista de contactos “añadir contacto” se desplegará una pestaña en la que el usuario ingresara datos del contacto y así poder registrarlo.
Autor	Montoya Dueñas Eduardo
Entradas	Nombre, apellido paterno, apellido materno, correo electrónico, número telefónico, ocupación.
Salidas	Mensaje de “Usuario registrado exitosamente” Mensaje de error “Usuario ya registrado”

	Mensaje de error "Campos faltantes"
Restricciones	Los datos deben cumplir las condiciones de cada campo (correo, límite de caracteres), si estas no cumplen con lo indicado el contacto no se registrará de forma correcta.
Actor	Usuario

Figura 1.11 Requerimiento funcional 10

ID Requisito	RF11
Nombre	Enviar correo electrónico
Descripción	El usuario podrá enviar un correo mediante el botón de "Enviar correo" al contacto que haya seleccionado mediante la lista, o bien, podrá ingresar manualmente el correo electrónico.
Autor	Montoya Dueñas Eduardo
Entradas	Correo electrónico, asunto, cuerpo del correo
Salidas	Mensaje de "Enviando correo" Mensaje de "Correo enviado" Mensaje de "Cuerpo del correo vacío" Mensaje de "Contacto no encontrado"
Restricciones	El correo electrónico debe coincidir con el contacto registrado y el cuerpo del correo debe tener contenido, si no se cumple saldrá un mensaje de error.
Actor	Usuario

Figura 1.12 Requerimiento funcional 11

ID Requisito	RF12
Nombre	Registrar productos
Descripción	El usuario podrá registrar en inventario los productos que tendrá en venta mediante el botón "Agregar producto".

Autor	Vera Hernández Ian Daniel
Entradas	Nombre producto, marca producto, cantidad actual, caducidad, fecha recepción, contenido por unidad, unidad, precio por unidad, moneda.
Salidas	Mensaje de "Producto registrado" Mensaje de "Campos faltantes" Mensaje de "Producto ya registrado"
Restricciones	Todos los campos deben estar llenos (a excepción de caducidad que es opcional) y el producto no debe haber sido anteriormente registrado, si no se cumplen estas condiciones saldrá un mensaje de error.
Actor	Usuario

Figura 1.13 Requerimiento funcional 12

ID Requisito	RF13
Nombre	Borrar productos
Descripción	El usuario podrá borrar el producto que guste seleccionar.
Autor	Vera Hernandez Ian Daniel
Entradas	Botón de borrar
Salidas	Mensaje de confirmación Mensaje de "Artículo borrado"
Restricciones	Si el usuario no presiona el botón de borrar no se eliminará el producto.
Actor	Usuario

Figura 1.14 Requerimiento funcional 13

ID Requisito	RF14
Nombre	Actualizar producto

Descripción	El usuario podrá actualizar los datos del producto que elija.
Autor	Reyes Romano Irvin David
Entradas	Nombre producto, fecha recepción, contenido por unidad, unidad, precio por unidad, moneda.
Salidas	Mensaje de "Producto actualizado" Mensaje de "Campos faltantes"
Restricciones	Todos los campos deben estar llenos, si no se cumplen estas condiciones saldrá un mensaje de error.
Actor	Usuario

Figura 1.15 Requerimiento funcional 14

ID Requisito	RF15
Nombre	Registrar tarea
Descripción	El usuario podrá registrar tareas en el apartado "Tareas", al pulsar el botón "Registrar tareas" el usuario deberá llenar los datos de la actividad (Día de inicio, día límite, Nombre de la actividad, hora de inicio, hora de finalización, descripción, prioridad de la tarea y responsable de la tarea), una vez finalizado esto deberá pulsar el botón "Aceptar" en la parte central inferior
Autor	Sanchez Aquino Alexis
Entradas	Información de la tarea
Salidas	Mensaje de confirmación Mensaje de "Operación cancelada"
Restricciones	El usuario deberá confirmar la creación de dicha tarea
Actor	Usuario

Figura 1.16 Requerimiento funcional 15

ID Requisito	RF16
Nombre	Borrar tarea

Descripción	El usuario podrá borrar la tarea que desee seleccionar.
Autor	Sanchez Aquino Alexis
Entradas	Botón de eliminar tarea.
Salidas	Mensaje de confirmación Mensaje de "Tarea eliminada" Mensaje de "Operación cancelada"
Restricciones	El usuario deberá confirmar la eliminación mediante el mensaje de confirmación, si no cumple con esto la operación será cancelada.
Actor	Usuario

Figura 1.17 Requerimiento funcional 16

ID Requisito	RF17
Nombre	Completar tarea.
Descripción	El usuario podrá completar las tareas mediante el botón de completar tarea.
Autor	Gonzalez Camacho Cristian Alejandro
Entradas	Botón de completar tarea
Salidas	Mensaje de confirmación Mensaje de "Tarea completada" Mensaje de "Operación cancelada"
Restricciones	El usuario podrá desmarcar la opción cuando lo desee en caso de un error
Actor	Usuario

Figura 1.18 Requerimiento funcional 17

ID Requisito	RF18
Nombre	Selección Gráficas de balance

Descripción	En la interfaz “Balance” se mostrarán gráficas que muestren las ganancias y pérdidas del mes (Dejando en la parte inferior el tiempo y la parte lateral izquierda el valor de la escala en dinero). También habrá una lista desplegable para seleccionar el mes a mostrar
Autor	Reyes Romano Irvin David
Entradas	Ventas mensuales agrupadas en semanas
Salidas	Gráfico “Balance”
Restricciones	Si no hay datos el gráfico no aparecerá Si el año está inconcluso, es decir, no ha llegado a los 12 meses en total la lista desplegable deberá limitarse a mostrar solo los datos de los meses registrados
Actor	Usuario

Figura 1.19 Requerimiento funcional 18

ID Requisito	RF19
Nombre	Impresión de gráficas
Descripción	En la interfaz “Balance” se mostrará un botón en la parte superior derecha de las gráficas, el cual debe permitir al usuario imprimir la gráfica seleccionada
Autor	Reyes Romano Irvin David
Entradas	Gráfica seleccionada
Salidas	Interfaz predeterminada de impresión
Restricciones	Si no hay datos el gráfico no se puede imprimir

	Se respetará el formato mostrado en las gráficas
Actor	Usuario

Figura 1.20 Requerimiento funcional 19

1.5.2 Plantillas SRC Para la definición de requisitos (No Funcionales)

ID Requisito	RNF 1
Nombre	Rendimiento de la pantalla principal
Descripción	El sistema debe renderizar la pantalla principal en un tiempo menor a 1 segundo bajo una carga de 50 usuarios concurrentes
Autor	Sanchez Aquino Alexis
Entradas	Ninguna
Salidas	Pantalla principal
Restricciones	Conexión mínima de 5 Mbps y hardware con al menos 4GB de RAM
Actor	Sistema

Figura 1.21 Requerimiento no funcional 1

ID Requisito	RNF 2
Nombre	Visualización de datos principales
Descripción	Los datos del sistema se deben actualizar automáticamente al momento de la entrada a la pantalla principal del usuario o al entrar a cualquier pestaña
Autor	Montoya Dueñas Eduardo
Entradas	Datos generales (Total de clientes, Clientes en esta semana, Clientes hoy, Porcentaje de aumento de clientela, Llamadas recibidas en total, Llamadas recibidas hoy, Estado el inventario, Tareas actuales, Tareas completadas, Tareas totales, Hora, Fecha, Eventos Marcados en el Calendario)
Salidas	Pantalla principal con datos actualizados
Restricciones	Si no se puede obtener un dato, se mostrará uno de los tres mensajes: "No hay acceso a Internet", "No hay registros de una empresa" o "Hubo un problema al obtener uno o varios datos"
Actor	Sistema

Figura 1.22 Requerimiento no funcional 2

ID Requisito	RNF 3
Nombre	Tiempo de actividad

Descripción	El sistema debe estar disponible el 99% del tiempo que esté abierto
Autor	Gonzalez Camacho Cristian Alejandro
Entradas	Ninguna
Salidas	Pantalla principal
Restricciones	Si el sistema no tiene acceso a internet, no estará disponible para el uso del cliente
Actor	Sistema

Figura 1.23 Requerimiento no funcional 3

ID Requisito	RNF 4
Nombre	Interfaz fácil de usar
Descripción	El sistema debe permitir a un usuario nuevo ser capaz de completar el registro de un cliente en menos de 3 minutos sin asistencia externa
Autor	Reyes Romano Irvin David
Entradas	Ninguna
Salidas	Pantalla principal
Restricciones	Ninguna
Actor	Sistema

Figura 1.24 Requerimiento no funcional 4

ID Requisito	RNF 5
Nombre	Limitación de lenguajes de programación
Descripción	El sistema debe ser programado única y exclusivamente con Java para un mejor manejo de funciones, procesamiento de información y compatibilidad de equipos
Autor	Sanchez Aquino Alexis
Entradas	Codificación del sistema
Salidas	Sistema
Restricciones	Las consultas en SQL se omiten como lenguaje externo a java ya que son esenciales para el funcionamiento de la aplicación y no interfieren con el núcleo de java

Actor	Sistema
-------	---------

Figura 1.25 Requerimiento no funcional 5

ID Requisito	RNF 6
Nombre	Facilidad de modificación del sistema
Descripción	El sistema debe ser de código abierto para que cualquiera tenga acceso a modificarlo y añadir, quitar o mejorar funciones del sistema
Autor	Montoya Dueñas Eduardo
Entradas	Codificación del sistema
Salidas	Sistema
Restricciones	Ninguna
Actor	Sistema

Figura 1.26 Requerimiento no funcional 6

ID Requisito	RNF 7
Nombre	Escalabilidad del sistema
Descripción	El sistema debe poder operar con al menos 100 usuarios activos o 10,000 usuarios de forma inactiva
Autor	Vera Hernandez Ian Daniel
Entradas	Inicios de sesión
Salidas	Sistema
Restricciones	Ninguna
Actor	Sistema

Figura 1.27 Requerimiento no funcional 7

ID Requisito	RNF 8
Nombre	Documentación técnica para colaboradores

Descripción	Debido a que el sistema es de código abierto, se debe proporcionar documentación técnica detallada (comentarios en código y manuales) para facilitar la modificación y mejora de funciones por parte de terceros
Autor	Vera Hernandez Ian Daniel
Entradas	Código fuente del sistema en Java
Salidas	Manual técnico y código fuente documentado
Restricciones	La documentación debe estar redactada de forma clara y seguir estándares de codificación
Actor	Sistema

Figura 1.28 Requerimiento no funcional 8

ID Requisito	RNF 9
Nombre	Bloqueo del sistema
Descripción	El sistema debe bloquear la cuenta tras 5 intentos fallidos de inicio de sesión por un periodo de 15 minutos
Autor	Reyes Romano Irvin David
Entradas	Intentos de inicios de sesión fallidos
Salidas	Alerta de intentos fallidos
Restricciones	Si la contraseña es incorrecta o le faltan caracteres o la cuenta es inexistente, mostrará un mensaje de alerta
Actor	Sistema

Figura 1.29 Requerimiento no funcional 9

ID Requisito	RNF 10
Nombre	Rendimiento del inicio de sesión
Descripción	El proceso de autenticación no debe tardar más de 1 minuto desde que el usuario hace clic en 'Ingresar'
Autor	Gonzalez Camacho Cristian Alejandro
Entradas	Intento de inicio de sesión
Salidas	Pantalla principal
Restricciones	Si el usuario no tiene señal de internet no podrá ingresar ni manualmente
Actor	Sistema

Figura 1.30 Requerimiento no funcional 10

ID Requisito	RNF 11
Nombre	Escalabilidad del sistema
Descripción	El sistema debe soportar al menos 500 inicios de sesión simultáneos por minuto sin

	degradación del servicio
Autor	Vera Hernandez Ian Daniel
Entradas	Intentos de inicio de sesión
Salidas	Pantalla principal
Restricciones	Ninguna
Actor	Sistema

Figura 1.31 Requerimiento no funcional 11

ID Requisito	RNF 12
Nombre	Indisponibilidad de servicios
Descripción	En caso de indisponibilidad del proveedor externo, el sistema debe ser capaz de mostrar un mensaje informativo claro en menos de 3 segundos, permitiendo al usuario intentar otros métodos de acceso si están configurados
Autor	Montoya Dueñas Eduardo
Entradas	Intentos de inicio de sesión
Salidas	Pantalla principal
Restricciones	En caso de no tener cuenta de GitHub o Google, se deberá crear una cuenta de las mismas para poder hacer el inicio de sesión
Actor	Sistema

Figura 1.32 Requerimiento no funcional 12

CAPÍTULO 2. DISEÑO DE INTERFACES GRÁFICAS DE USUARIO

En este capítulo se describe el diseño de la base de datos y las interfaces de la aplicación de escritorio

2.1 Desarrollo de la Base de datos

A continuación, mostraremos el proceso para la elaboración de la base de datos que se utilizó para el sistema CRM-Tec

2.1.1 Diseño

En esta sección, se muestra cómo está compuesta la base de datos, su estructura incluyendo los campos de cada tabla, mostrando la figura 2.1, en la cual se muestra la relación de los datos y los atributos que contendrá cada tabla.



Figu

Figura 2.1 Tablas y Relaciones.



Figura 2.2 Tablas y Relaciones.

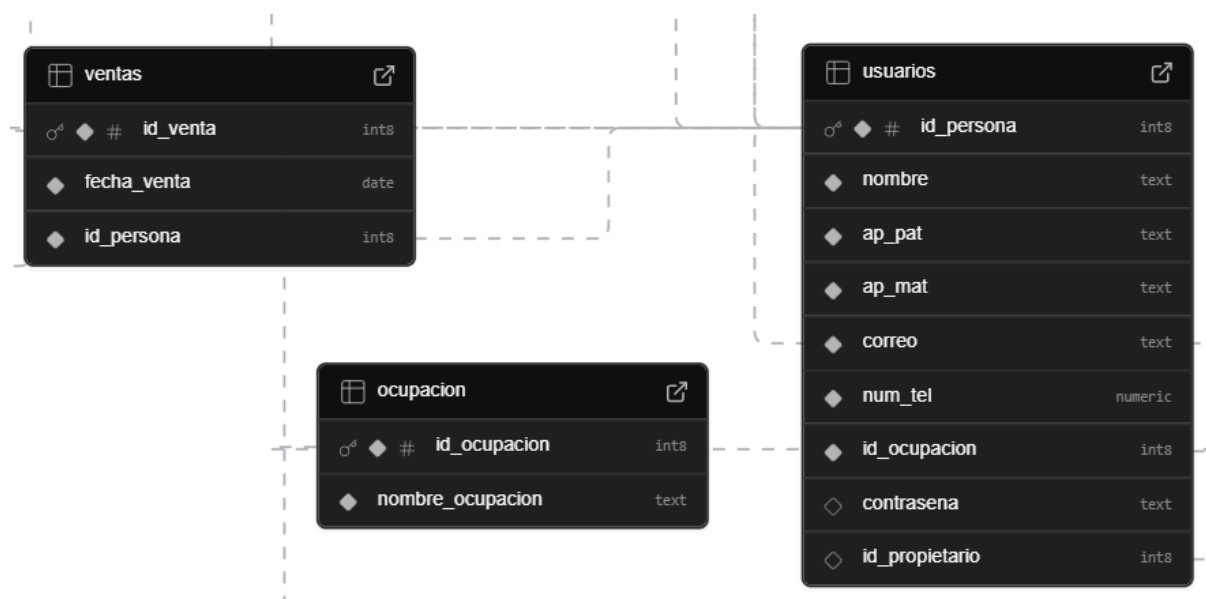


Figura 2.3 Tablas y Relaciones.



Figura 2.4 Tablas y Relaciones.

2.1.2 Diccionario de datos

Un diccionario de datos es un repositorio centralizado de metadatos que define, describe y estructura los elementos dentro de una base de datos. Documenta tablas, campos (tipos, longitud), relaciones, restricciones y el significado de los datos, garantizando coherencia, estandarización y una mejor interpretación para usuarios y desarrolladores. A continuación, se muestra el diccionario de datos correspondiente a la base de datos ver figura 2.5

Ocupación

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_ocupacion (PK)	bigint(8)	No				
nombre_ocupacion	Text	No				

Figura 2.5 Tabla Diccionario de la tabla Ocupación

La figura 2.6 muestra el índice de la tabla ocupación

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	Id_ocupacion	0	N/A	No

Figura 2.6 índice de la tabla Ocupación

La figura 2.7 muestra el Diccionario de datos de la tabla Productos

Productos

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_producto (PK)	bigint(8)	No				
nombre_prod	text	No				

marca	text	No				
caducidad	date	Si				
precio	double	No				
fecha_recepcion	date	Si				
id_persona (FK)	bigint(8)	Si				

Figura 2.7 Tabla Diccionario de la tabla Productos

La figura 2.8 muestra el índice de la tabla Productos

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_producto	0	N/A	No

Figura 2.8 índice de la tabla Productos

La figura 2.9 muestra el Diccionario de datos de la tabla Usuarios

Usuarios

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_persona (PK)	bigint(8)	No				
nombre	text	No				
ap_pat	text	No				

ap_mat	Text	No				
correo	Text	No				
num_tel	numeric	No				
contrasena	text	Si				
id_propietario	bigint(8)	Si				
id_ocupacion (FK)	bigint(8)	No		Ocupacion → id_ocupacion		

Figura 2.9 Tabla Diccionario de la tabla Usuarios

La figura 2.10 muestra el índice de la tabla Usuarios

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_persona	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_ocupacion	0	N/A	No
FOREIGN	BTREE	No	N/A	id_propietario	0	N/A	Si

Figura 2.10 Índice de la tabla Usuarios

La figura 2.11 muestra el Diccionario de datos de la tabla Stock

Stock

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_stock (PK)	bigint(8)	No				
cant_exist	Numeric	No				
contenido	Numeric	Si				
unidad	text	Si				
id_producto (FK)	numeric	No		Productos → id_producto		

Figura 2.11 Tabla Diccionario de la tabla Stock

La figura 2.12 muestra el índice de la tabla Stock

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_stock	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_producto	0	N/A	No

Figura 2.12 índice de la tabla Stock

La figura 2.13 muestra el Diccionario de datos de la tabla Administrador

Administrador

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_admin (PK)	bigint(8)	No				

curp	varchar	No				
rfc	varchar	No				
id_persona (FK)	bigint(8)	No		usuarios → id_persona		

Figura 2.13 Tabla Diccionario de la tabla Administrador

La figura 2.14 muestra el índice de la tabla Administrador

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_admin	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_persona	0	N/A	No

Figura 2.14 índice de la tabla Administrador

La figura 2.15 muestra el Diccionario de datos de la tabla Ventas

Ventas

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_venta (PK)	bigint(8)	No				
Fecha_venta	Date	No				
id_persona (FK)	bigint(8)	No		usuarios → id_persona		

Figura 2.15 Tabla Diccionario de la tabla Ventas

La figura 2.16 muestra el índice de la tabla Ventas

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_venta	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_persona	0	N/A	No

Figura 2.16 índice de la tabla Ventas

La figura 2.17 muestra el Diccionario de datos de la tabla Provee

Provee

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_provee (PK)	bigint(8)	No				
Fecha_entrega	Date	No				
Cant_com	numeric	No				
Id_producto (FK)	bigint(8)	No		Productos → id_producto		
id_persona (FK)	bigint(8)	No		usuarios → id_persona		

Figura 2.18 Tabla Diccionario de la tabla Provee

La figura 2.19 muestra el índice de la tabla Provee

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_provee	0	0	No
FOREIGN	BTREE	No	N/A	Id_producto	0	0	No
FOREIGN	BTREE	No	N/A	Id_persona	0	0	No

Figura 2.19 índice de la tabla Provee

La figura 2.20 muestra el Diccionario de datos de la tabla Negocio

Negocio

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_negocio (PK)	bigint(8)	No				
Nombre_negocio	text	No				
id_admin (FK)	bigint(8)	No		administrador → id_admin		

Figura 2.20 Tabla Diccionario de la tabla Negocio

La figura 2.21 muestra el índice de la tabla Negocio

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_provee	0	N/A	No

FOREIGN	BTREE	No	N/A	Id_admin	0	N/A	No
---------	-------	----	-----	----------	---	-----	----

Figura 2.21 índice de la tabla Negocio

La figura 2.22 muestra el Diccionario de datos de la tabla negocio_balance

Negocio_balance

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_balance (PK)	bigint(8)	No				
Saldo	numeric	No				
Gastos	numeric	No				
perdidas	Numeric	No				
ganancias	numeric	No				
id_negocio (FK)	bigint(8)	No		Negocio → id_negocio		
id_persona (FK)	bigint(8)	No		Usuarios → id_persona		

Figura 2.22 Tabla Diccionario de la tabla negocio_balance

La figura 2.23 muestra el índice de la tabla negocio_balance

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_balance	0	N/A	No

FOREIGN	BTREE	No	N/A	Id_negocio	0	N/A	No
---------	-------	----	-----	------------	---	-----	----

Figura 2.23 índice de la tabla negocio_balance

La figura 2.24 muestra el Diccionario de datos de la tabla negocio_direccion

Negocio_direccion

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_direccion (PK)	bigint(8)	No				
calle	Text	No				
colonia	text	No				
codpos	Bigint(8)	No				
municipio	text	No				
estado	Text	No				
id_negocio (FK)	bigint(8)	No		Negocio → id_negocio		

Figura 2.24 Tabla Diccionario de la tabla negocio_direccion

La figura 2.25 muestra el índice de la tabla negocio_direccion

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_direccion	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_negocio	0	N/A	No

--	--	--	--	--	--	--	--

Figura 2.25 Índice de la tabla negocio_direccion

La figura 2.26 muestra el Diccionario de datos de la tabla trabaja

trabaja

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_trabaja (PK)	bigint(8)	No				
Id_persona (PK)	Bigint(8)	No		usuarios → id_persona		
id_negocio (FK)	bigint(8)	No		Negocio → id_negocio		

Figura 2.26 Tabla Diccionario de la tabla trabaja

La figura 2.27 muestra el índice de la tabla trabaja

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_trabaja	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_persona	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_negocio	0	N/A	No

Figura 2.27 Índice de la tabla trabaja

La figura 2.28 muestra el Diccionario de datos de la tabla venta_detalle

Venta_detalle

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_detalle (PK)	bigint(8)	No				
Cant_vend	numeric	No				
Prec_fin	numeric	No				
Id_venta (FK)	bigint(8)	No		Ventas → id_venta		
id_producto (FK)	bigint(8)	No		productos → id_producto		

Figura 2.28 Tabla Diccionario de la tabla venta_detalle.

La figura 2.29 muestra el índice de la tabla venta_detalle

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id_detalle	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_venta	0	N/A	No
FOREIGN	BTREE	No	N/A	Id_producto	0	N/A	No

Figura 2.29 índice de la tabla venta_detalle.

La figura 2.30 muestra el Diccionario de datos de la tabla tareas

tareas

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id_tarea (PK)	bigint(8)	No				
nombre_actividad	varchar	No				
fecha_limite	date	Si				
hora_inicio	varchar	No				
hora_fin	varchar	Si				
descripcion	text	Si				
prioridad	varchar	No				
responsable	varchar	No				
completada	boolean	Si				
created_at	timestampz	Si				
id_usuario(FK)	bigint(8)	No		Usuarios -> id_persona		

Figura 2.30 Tabla Diccionario de la tabla tareas

La figura 2.31 muestra el índice de la tabla tareas

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
-----------------	------	-------	-------------	---------	--------------	--------------	------

PRIMARY	BTREE	Si	N/A	id_tarea	0	N/A	No
FOREIGN	BTREE	No	N/A	id_usuario	0	N/A	No

Figura 2.31 índice de la tabla tareas

La figura 2.32 muestra el Diccionario de datos de la tabla eventos

eventos

Columna	Tipo	Nulo	Predeterminado	Enlaces	Comentarios	MIME
id(PK)	bigint(8)	No				
title	varchar	No				
start	date	Si				
id_persona(FK)	varchar	No		Usuarios -> id_persona		

Figura 2.32 Tabla Diccionario de la tabla eventos

La figura 2.33 muestra el índice de la tabla tareas

Índice

Nombre de clave	Tipo	Único	Empaquetado	Columna	Cardinalidad	Cotejamiento	Nulo
PRIMARY	BTREE	Si	N/A	id	0	N/A	No
FOREIGN	BTREE	No	N/A	id_persona	0	N/A	No

Figura 2.33 índice de la tabla tareas

2.1.2 Creación de la base de datos

En este apartado se muestra una parte del código (Figura 2.34), para la creación de la base de datos Mtec, mostrando la tabla Productos, con los campos que contiene y que serán almacenados en ella. Esta base de datos se creo con el gestor de Supabase.

```

1 create database Mtec;
2 \c Mtec;
3 |
4 create table public.productos (
5     id_producto bigint generated by default as identity not null,
6     nombre_prod text not null,
7     marca text not null,
8     caducidad date null,
9     precio double precision not null,
10    fecha_recepcion date null,
11    contenido numeric not null,
12    unidad text not null,
13    constraint productos_pkey primary key (id_producto)
14 ) TABLESPACE pg_default;
```

Figura 2.34 Código para crear la base de datos.

2.2 Desarrollo de interfaces

En este apartado se muestra parte por parte cada una de las pantallas en la aplicación , así como su función, con la finalidad que el usuario comprenda como fue diseñada y la usabilidad de cada una de estas. En la figura 2.35 se muestra el diseño de la pantalla Login, que es el ingreso del usuario a la aplicación de escritorio.

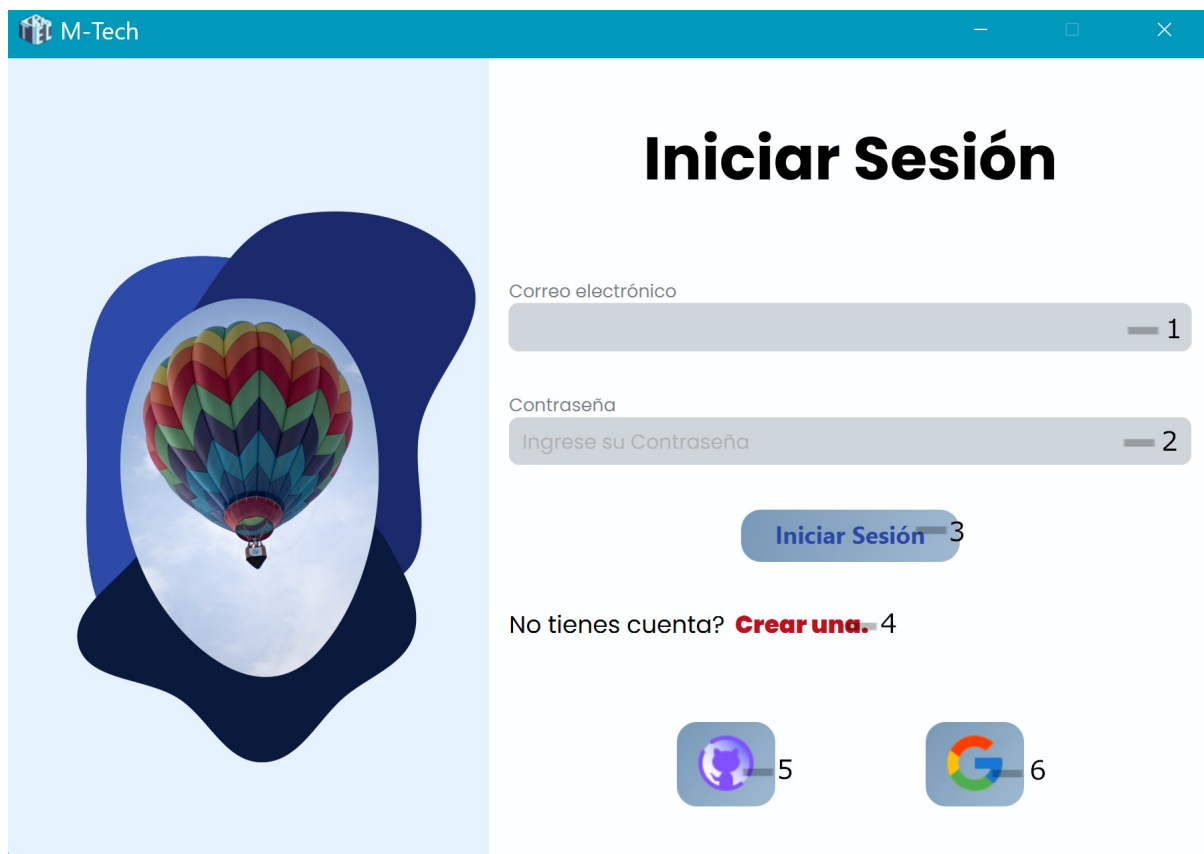


Figura 2.35 Interfaz de Login.

En la figura 2.36 se muestra la pantalla principal, la interfaz gráfica que el usuario podrá ver al iniciar sesión.

1. Campo de texto para el correo
2. Campo de texto para la contraseña
3. Botón de iniciar sesión
4. Link que redirige al usuario a la pantalla de crear cuenta
5. Botón de “iniciar sesión con GitHub” que muestra el logo de GitHub
6. Botón de “Iniciar sesión con Google” que muestra el logo de Google

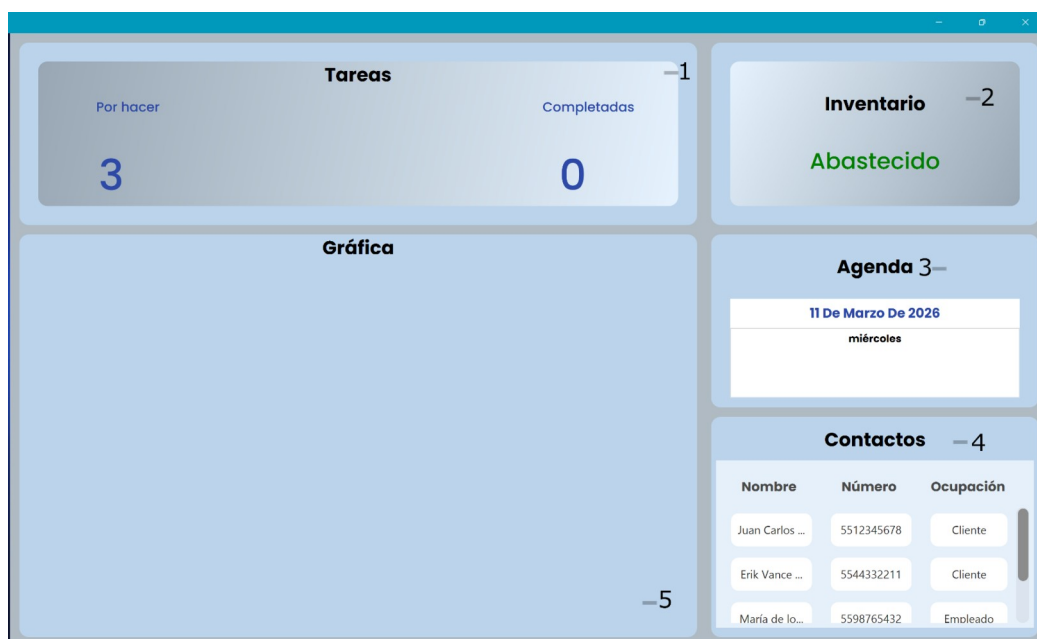


Figura 2.36 Interfaz Principal.

En la figura 2.37 se muestra la pantalla de tareas, la interfaz gráfica de la lista de las tareas registradas, así como las opciones de borrar y completar tareas.

1. Widget "Tareas" que muestra las etiquetas "Tareas por hacer" y "Tareas completadas"
2. Widget "Inventario" que muestra la etiqueta "Abastecido" y "Falta abastecer"
3. Widget "Agenda" que muestra la tabla "Evento próximo" contiene la etiqueta "fecha actual" Como encabezado y como subtítulo "Día" y la etiqueta "Evento"
4. Widget "Contactos" que muestra la tabla "Contactos" Contiene las columnas "Nombre", "Número" y "Ocupación"
5. Widget de Gráfica que muestra la gráfica "Balance"

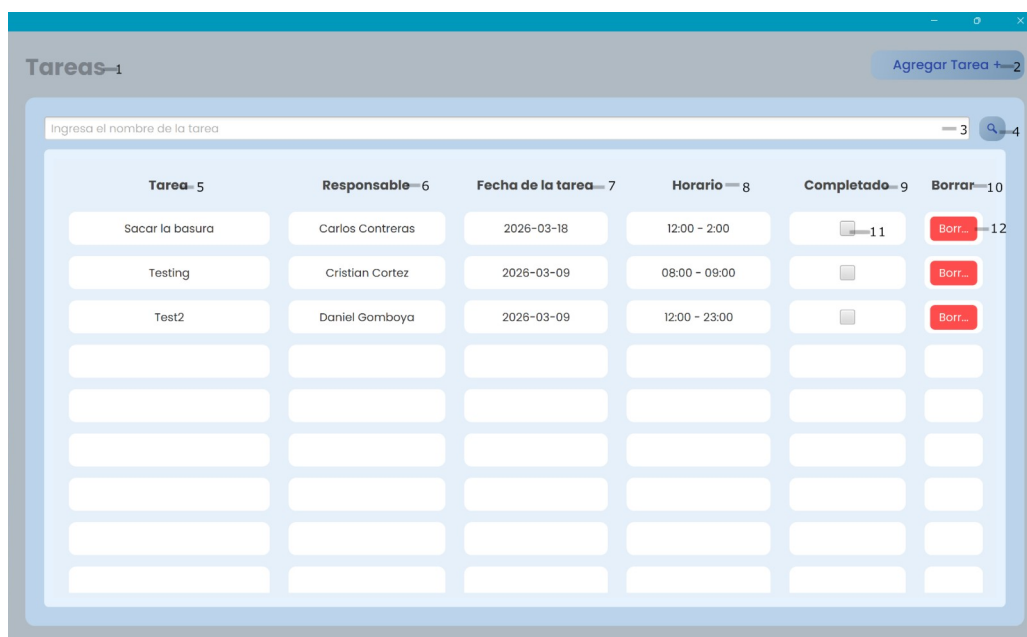


Figura 2.37 Interfaz Principal de Tareas.

En la figura 2.37 Se muestra la pantalla de tareas, la interfaz gráfica que se va a mostrar al ingresar al apartado de tareas, está encargada de mostrar, registrar, borrar y actualizar las tareas del usuario.

1. Etiqueta Tareas
2. Botón Agregar tarea que redirige al usuario al formulario para agregar una Tarea
3. Barra de búsqueda para encontrar registros en la tabla "Tareas"
4. Botón "búsqueda" para ejecutar la búsqueda
5. Columna "Tarea"
6. Columna "Responsable"
7. Columna "Fecha de la tarea"
8. Columna "Horario"
9. Columna "Completado"
10. Columna "Borrar"
11. Casilla "Completado" o "Sin completar"
12. Botón "Borrar Tarea"

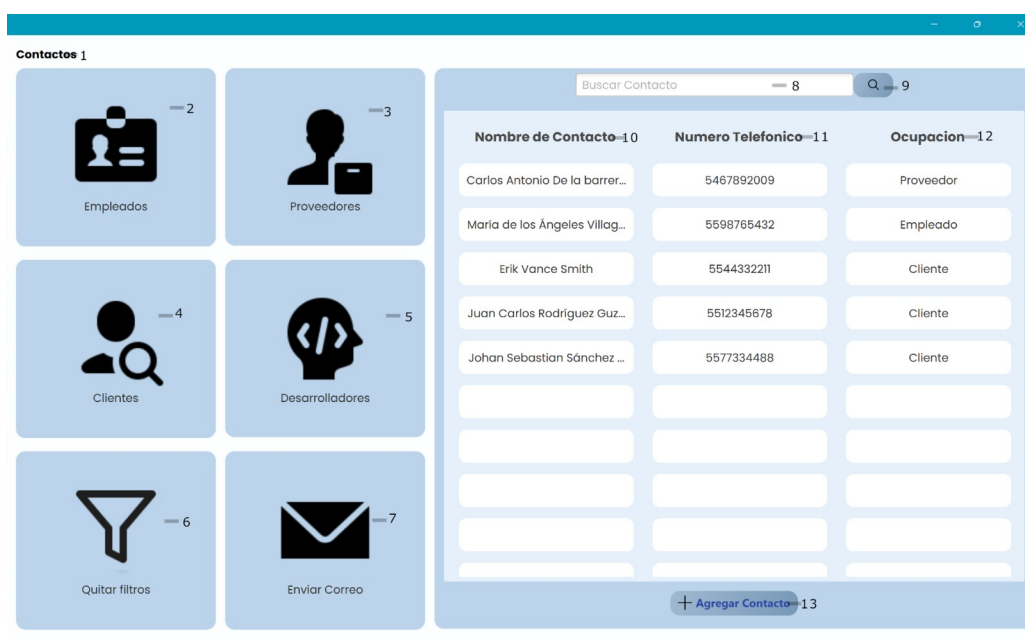


Figura 2.38 Interfaz Principal de Contactos.

En la figura 2.38 se muestra la pantalla de contactos, la interfaz gráfica que se va a mostrar al ingresar al apartado de contactos, está encargada de ver los contactos registrados, poder enviar correos electrónicos, agregar contactos y buscar contactos.

1. Etiqueta contactos
2. Botón “Empleados” filtra el contenido de la tabla “Contactos” por la ocupación “Empleado”
3. Botón “Proveedores” filtra el contenido de la tabla “Contactos” por la ocupación “Proveedor”
4. Botón “Clientes” filtra el contenido de la tabla “Contactos” por la ocupación “Clientes”
5. Botón “Desarrolladores” filtra el contenido de la tabla “Contactos” por la ocupación “Desarrollador”
6. Botón “Quitar filtros” quita los filtros aplicados a la tabla
7. Botón “Enviar correo” envía al usuario al formulario para enviar un correo al contacto
8. Barra de búsqueda para encontrar registros en la tabla “Contactos”
9. Botón “Buscar” para ejecutar la búsqueda en la tabla “Contactos”
10. Columna “Nombre de contacto”
11. Columna “Número telefónico”
12. Columna “Ocupación”

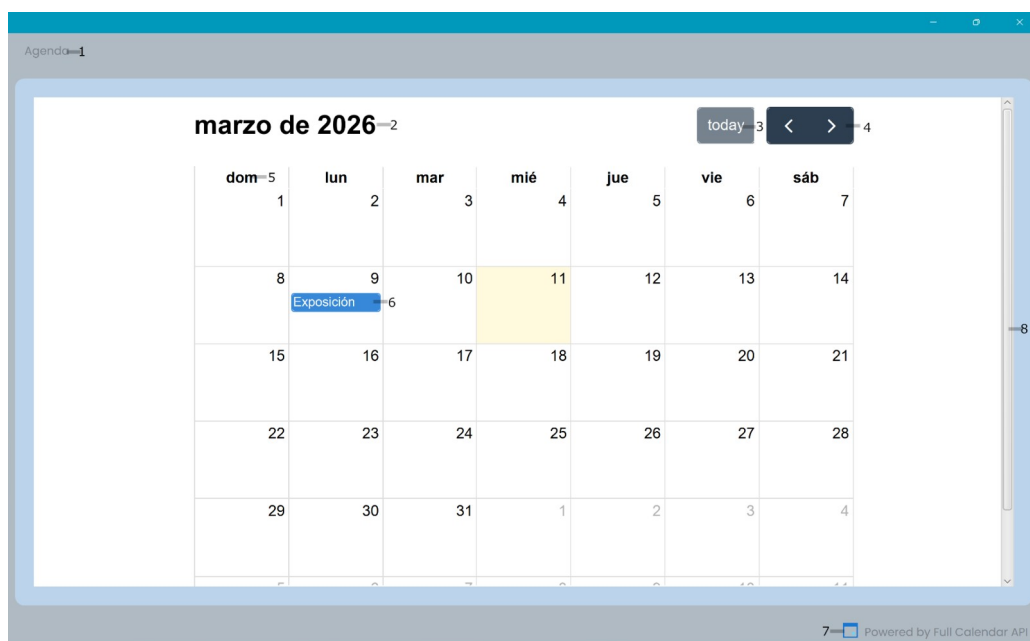


Figura 2.39 Interfaz de Agenda.

En la siguiente figura 2.39 se muestra el maquetado desarrollado de la interfaz dedicada a la Agenda donde se mostrarán todos los días con y sin eventos marcados durante el mes. En la cual se muestra en los días con evento el título de dicho evento, ya sea seleccionado de las opciones predeterminadas o editado por el administrador mismo.

1. Etiqueta “Agenda”
2. Etiqueta “Fecha actual”
3. Botón “selector de fecha”
4. Botones de movilidad
5. Día hábil
6. Día de evento
7. Etiqueta de API “FullCalendar”



Figura 2.40 Interfaz de Inteligencia Artificial.

En la siguiente figura 2.40 se muestra el maquetado desarrollado de la interfaz de un chat con una IA (Gemini), en la cual se recibirá el mensaje del usuario para posteriormente la IA generar una respuesta a la pregunta/duda/comentario del usuario.

1. Etiqueta “Mensaje de bienvenida”
2. Contenedor “Mensaje del usuario”
3. Contenedor “Respuesta de la IA”
4. Barra de texto “Mensaje del usuario”
5. Botón “Enviar”
6. Etiqueta de API “Gémini”

CAPÍTULO 3. DISEÑO E IMPLEMENTACIÓN DE LOS MÓDULOS

Un elemento esencial para la guía del entendimiento de grandes proyectos es la creación de elementos explicativos y metódicos mediante el Lenguaje Unificado de Modelado (UML), es un lenguaje estándar que se utiliza para visualizar, especificar, construir y documentar los componentes de un sistema de software.

A diferencia de los lenguajes de programación como Java o C#, UML no es código; es un conjunto de herramientas gráficas (diagramas) que ayudan a los desarrolladores y arquitectos a "dibujar" los planos de una aplicación antes de empezar a programar.

Para el correcto desarrollo de las funcionalidades y diseños del sistema a realizar, se debió tener de antemano una guía o mapa de seguimiento del curso del funcionamiento de una actividad, por lo cual se optó por la idea de manejar una serie de Diagramas de Actividad, los cuales darán una explicación visual y metodológica del camino del funcionamiento de los módulos realizados.

En la figura 3.1 se muestra el Diagrama de Actividad-Inventario, se muestra el cómo este apartado dará una vista de un producto al poder registrarlo dentro de la base de datos.

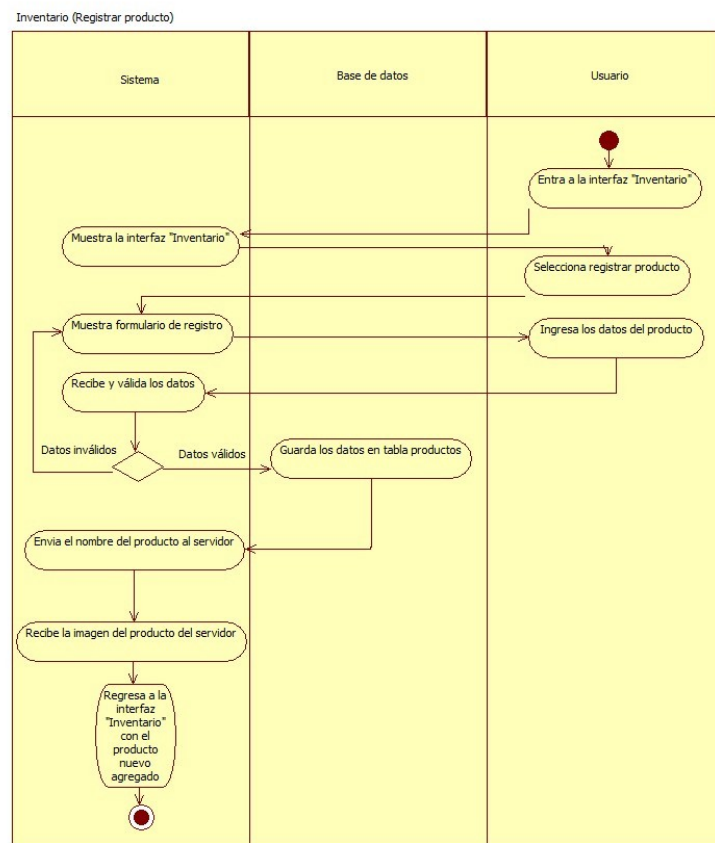


Figura 3.1 Diagrama de Actividad-Inventario

En la figura 3.2 se muestra el Diagrama de Actividad-Agenda, se muestra el funcionamiento mismo de la agenda desarrollada, en la cual se pueden ver eventos a lo largo del mes y se podrán registrar dichos eventos por el usuario.

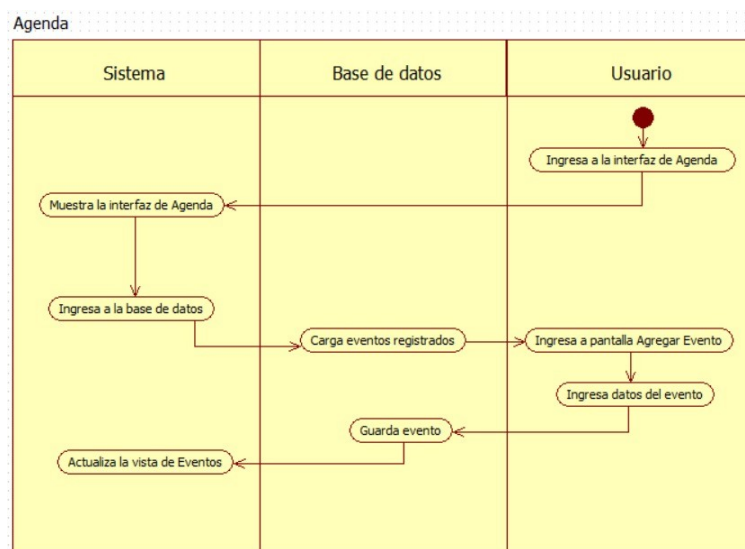


Figura 3.2 Diagrama de Actividad-Agenda

En la figura 3.3 se muestra el Diagrama de Actividad-Tareas, se muestra el cómo se mostrarán las tareas existentes a desarrollarse en el día, al igual que el registro de las mismas.

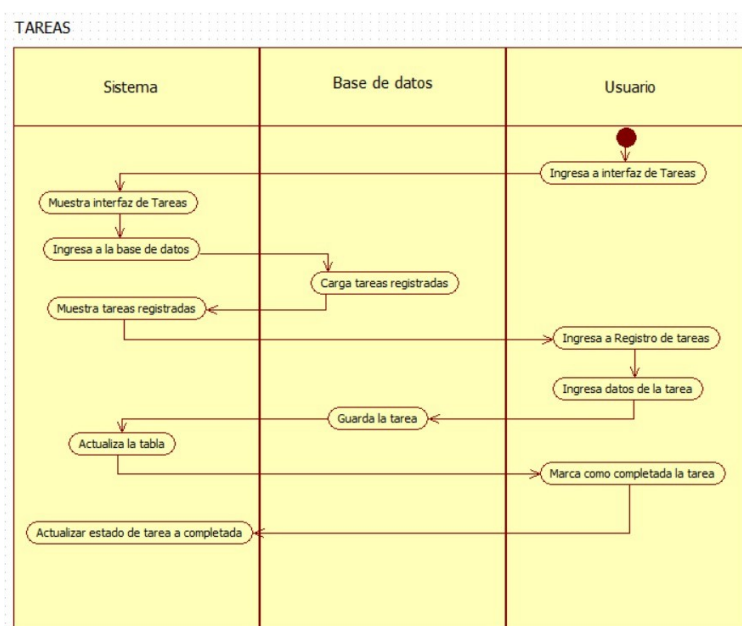


Figura 3.3 Diagrama de Actividad-Tareas

En la figura 3.4 se muestra el Diagrama de Actividad-Contacto, se muestra cómo se da la lista de contactos, sus filtros y el proceso para agregar un nuevo contacto.

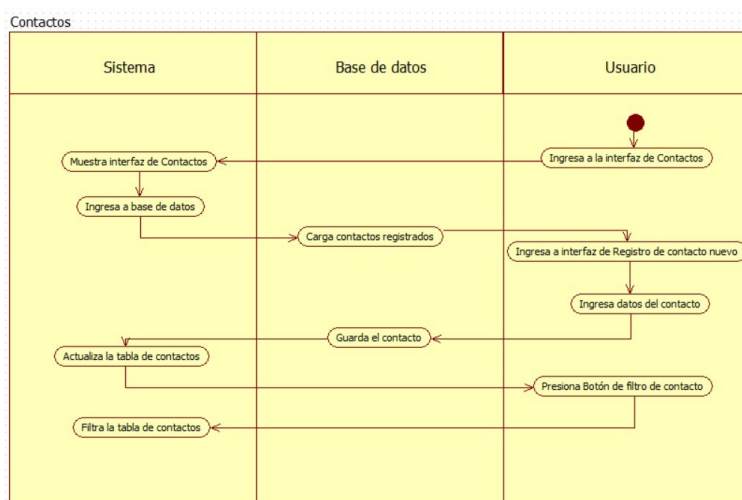


Figura 3.4 Diagrama de Actividad-Contacto

En la figura 3.5 se muestra el Diagrama de Actividad-Gráfica, mostrará la forma en que se deberán ingresar los datos requeridos para poder generar una visualización gráfica del estado del negocio.

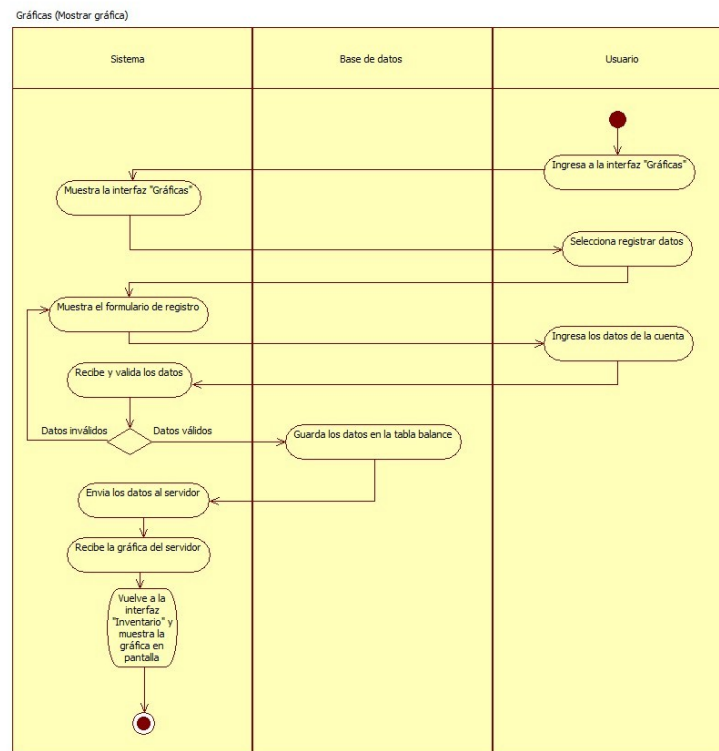
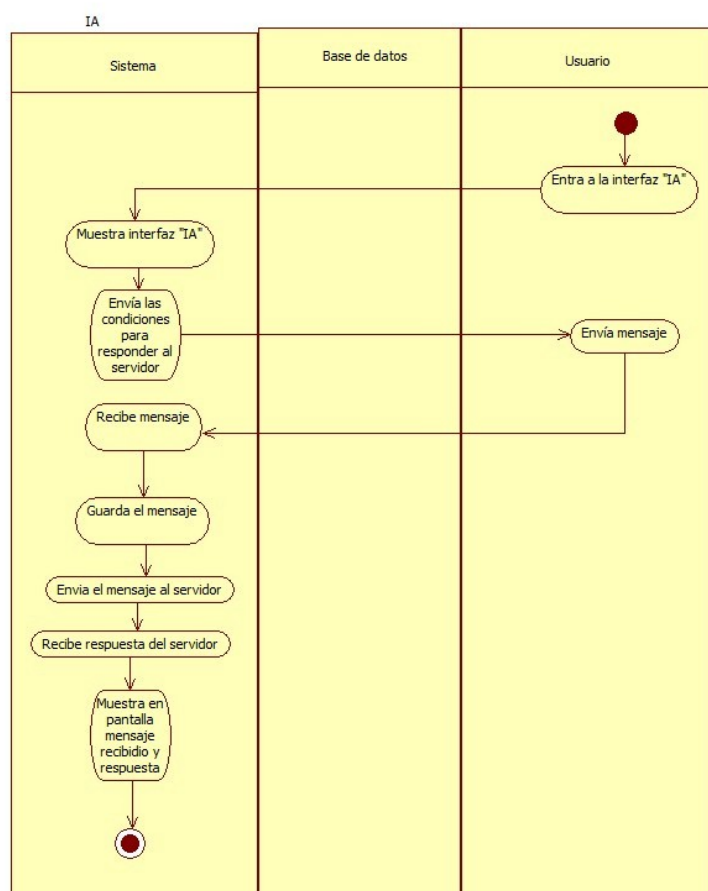


Figura 3.5 Diagrama de Actividad-Gráfica

En la figura 3.6 se muestra el Diagrama de Actividad-IA, se muestra el cómo se realiza el proceso de chat con la IA implementada en la aplicación para el apoyo de consejos del ámbito financiero .

**Figura 3.6** Diagrama de Actividad-IA

3.1 Funcionamiento del sistema

Una vez entendida la navegación del sistema mismo, es necesario adentrarse a lo que sería un funcionamiento a nivel de código del sistema mismo. Por lo cual, el propósito de este apartado es dar una introducción al cerebro mismo de cada apartado o sección diseñada, en la cual se busca dar a entender el cómo reacciona la lógica del código, las funciones que este maneja y por ende, el propósito que busca generar para el proyecto mismo.

Por consiguiente, se dará una breve explicación de la lógica de los módulos, llamados controladores, que manejan los procesos, variables y consumos para el correcto funcionamiento de dichos apartados.

1. Inventario

Este controlador gestiona la estructura de navegación de la vista de inventario. Se encarga de capturar los clics en el menú lateral para redirigir al usuario hacia las distintas áreas del sistema (Agenda, Tareas, Contactos, etc.) y habilita el botón para transitar a la pantalla de 'Agregar producto'.

```
try {
    conectar = BaseDatos.Conexion.conectar();
    conectar.setAutoCommit(autoCommit: false);

    String sqlProducto = "INSERT INTO productos (nombre_prod, marca, caducidad, precio, fecha_recepcion) VALUES (?, ?, ?, ?, ?) RETURNING id_producto";
    PreparedStatement psProd = conectar.prepareStatement(sqlProducto);
    psProd.setString(parameterIndex: 1, txtNombreProducto.getText());
    psProd.setString(parameterIndex: 2, txtMarca.getText());

    if (!txtCadAnio.getText().isEmpty() || !txtCadMes.getText().isEmpty() || !txtCadDia.getText().isEmpty()) {
        String fechacad = txtCadAnio.getText() + "-" + txtCadMes.getText() + "-" + txtCadDia.getText();
        psProd.setDate(parameterIndex: 3, Date.valueOf(fechacad));
    } else {
        psProd.setNull(parameterIndex: 3, Types.DATE);
    }

    psProd.setDouble(parameterIndex: 4, Double.parseDouble(txtPreUnidad.getText()));
    String fechaRec = txtRecAnio.getText() + "-" + txtRecMes.getText() + "-" + txtRecDia.getText();
    psProd.setDate(parameterIndex: 5, Date.valueOf(fechaRec));

    ResultSet rsID = psProd.executeQuery();
    long idNuevoProducto = 0;
    if (rsID.next()) {
        idNuevoProducto = rsID.getLong(columnLabel: "id_producto");
    }

    String sqlStock = "INSERT INTO stock (cant_exist, contenido, unidad, id_producto) VALUES (?, ?, ?, ?)";
    PreparedStatement psStock = conectar.prepareStatement(sqlStock);
    psStock.setDouble(parameterIndex: 1, Double.parseDouble(txtActual.getText()));
    psStock.setDouble(parameterIndex: 2, Double.parseDouble(txtContenido.getText()));
    psStock.setString(parameterIndex: 3, cmbUnidad.getValue() != null ? cmbUnidad.getValue() : "N/A");
    psStock.setLong(parameterIndex: 4, idNuevoProducto);

    psStock.executeUpdate();

    conectar.commit();
}
```

Figura 3.7 Controlador inventario

2. Agenda

El siguiente apartado corresponde a la interfaz visual de una agenda diseñada para la gestión de eventos del mes. La principal característica de esta sección es la incrustación de un calendario interactivo web (cargado desde un recurso HTML local), el cual se renderiza dinámicamente mediante un motor WebView dentro de un contenedor exclusivo. Actualmente, la interfaz establece la estructura base para la futura programación de las funciones de registro, actualización y eliminación de eventos.

```
private void cargarCalendarioWeb() {  
    try {  
        WebView webView = new WebView();  
        WebEngine webEngine = webView.getEngine();  
        webEngine.setJavaScriptEnabled(true);  
  
        URL url = getClass().getResource(name: "/resources/calendar.html");  
        if (url != null) {  
            webEngine.load(url.toExternalForm());  
        } else {  
            System.err.println("Error: No se encontró el archivo calendar.html");  
        }  
  
        VBox.setVgrow(webView, Priority.ALWAYS);  
        contCalendar.getChildren().clear();  
        contCalendar.getChildren().add(webView);  
    } catch (Exception e) {  
        System.err.println("Error al inicializar el calendario web: " + e.getMessage());  
        e.printStackTrace();  
    }  
}
```

Figura 3.8 Controlador Agenda

3. Tareas

Este elemento manejará una vista de tareas registradas por el mismo usuario, diferenciándose del apartado de la agenda al generar solamente actividades a realizarse durante el día. Estas tareas contienen un nombre, fecha, horario, si se ha completado o no y un apartado para poder eliminar la tarea si es que se desea eliminar el registro de dicha.


```

public class Controller_Riarea {
    void guardarTarea(ActionEvent event) {
        if (txtActividad.getText() == null || txtActividad.getText().isEmpty()) {
            System.out.println("Oye tu, eres tontito o que, no has escrito nada en el campo de actividad");
            return;
        }

        long idUsuarioActivo = SesionGlobal.getIdUsuarioActivo();
        if (idUsuarioActivo == 0) {
            System.out.println("No hay un usuario activo");
            return;
        }

        String prioridadSeleccionada = "";
        if (rdbAlta.isSelected())
            prioridadSeleccionada = "Alta";
        else if (rdbMedia.isSelected())
            prioridadSeleccionada = "Media";
        else if (rdbBaja.isSelected())
            prioridadSeleccionada = "Baja";

        try {
            Connection conectar = Conexion.conectar();
            String sql = "INSERT INTO tareas (nombre_actividad, fecha_limite, hora_inicio, hora_fin, descripcion, prioridad, responsable, id_usuario) VALUES (?, ?, ?, ?, ?, ?, ?, ?)";
            PreparedStatement ps = conectar.prepareStatement(sql);
            ps.setString(parameterIndex: 1, txtActividad.getText());
            if (dtFecha.getValue() != null) {
                ps.setDate(parameterIndex: 2, java.sql.Date.valueOf(dtFecha.getValue()));
            } else {
                ps.setNull(parameterIndex: 2, java.sql.Types.DATE);
            }
            ps.setString(parameterIndex: 3, txthoraInicio.getText());
            ps.setString(parameterIndex: 4, txthoraFin.getText());
            ps.setString(parameterIndex: 5, txtdescripcion.getText());
            ps.setString(parameterIndex: 6, prioridadSeleccionada);
            ps.setString(parameterIndex: 7, txtresponsable.getText());
            ps.setLong(parameterIndex: 8, idUsuarioActivo);
            ps.executeUpdate();
            conectar.close();
        }
    }
}

```

Figura 3.9 Controlador Tarea

4. Contacto

Este apartado maneja una lista de contactos, los cuales solicita para su agregación el nombre de dicho contacto, número, correo y la ocupación que este desempeña, en nuestro caso se dividieron entre 4 roles las ocupaciones: Empleado, proveedor, cliente y desarrollador. En este mismo apartado existe una barra de búsqueda la cual reacciona ante los nombre de los contactos, sus números de teléfono o sus roles; pero de igual manera se cuenta con 4 tarjetas a lado de la lista de contactos en la cual se puede filtrar los contactos por medio de sus roles.

```

public class Controller_AContacto {

    @FXML
    void guardarContacto(ActionEvent event) {
        if (txtNombre.getText().isEmpty() || txtAP.getText().isEmpty() || txtAM.getText().isEmpty() ||
            txtNum.getText().isEmpty() || cmbboxOcupacion.getValue() == null) {
            alerta(titulo: "Advertencia", mensaje: "Completa todos los datos del contacto", AlertType.WARNING);
            return;
        }

        int idOcupacion = 0;
        String seleccion = cmbboxOcupacion.getValue();

        switch (seleccion) {
            case "Administrador":
                idOcupacion = 1;
                break;
            case "Empleado":
                idOcupacion = 2;
                break;
            case "Proveedor":
                idOcupacion = 3;
                break;
            case "Cliente":
                idOcupacion = 4;
                break;
            case "Desarrollador":
                idOcupacion = 5;
            default:
                idOcupacion = 4;
        }

        try {
            Connection conectar = Conexion.conectar();

            String sql = "INSERT INTO usuarios (nombre, ap_pat, ap_mat, correo, num_tel, id_ocupacion) VALUES (?, ?, ?, ?, ?, ?)";
            PreparedStatement ps = conectar.prepareStatement(sql);
            ps.setString(parameterIndex: 1, txtNombre.getText());
            ps.setString(parameterIndex: 2, txtAP.getText());
        }
    }
}

```

Figura 3.10 Controlador Contacto

5. Gráfica

Este apartado como su nombre lo indica, busca proporcionar al usuario una vista de una gráfica de barras de las finanzas de la empresa del usuario, esto mediante la API de PayPal, que proporciona la información a la tabla dentro de la base de datos, para que posteriormente la gráfica solicite la información requerida para mostrar una vista de esta en la tabla del usuario

```

public class Controller_Grafica {

    private void iniciarGraficaTiempoReal() {
        Timeline timeline = new Timeline(new KeyFrame(
            Duration.seconds(5),
            event -> actualizarGraficaDesdeSupabase()
        ));
        timeline.setCycleCount(Timeline.INDEFINITE);
        timeline.play();

        actualizarGraficaDesdeSupabase();
    }

    private void actualizarGraficaDesdeSupabase() {
        new Thread(() -> {
            try {
                // 1. Petición a Supabase
                HttpRequest request = HttpRequest.newBuilder()
                    .uri(URI.create(SUPABASE_URL))
                    .header(name: "apikey", SUPABASE_KEY)
                    .header(name: "Authorization", "Bearer " + SUPABASE_KEY)
                    .GET()
                    .build();

                HttpResponse<String> response = httpClient.send(request, HttpResponse.BodyHandlers.ofString());
                String jsonResponse = response.body();

                // 2. Procesar el JSON y extraer etiquetas y valores
                String[] datosProcesados = procesarDatosSupabase(jsonResponse);
                String etiquetasParaChart = datosProcesados[0]; // Ej: "Ene", "Feb"
                String valoresParaChart = datosProcesados[1];    // Ej: 100,200

                // 3. Crear configuración QuickChart
                String chartConfig = "{ \"type\": \"bar\", \"data\": { \"labels\": [" + etiquetasParaChart + "], \"datasets\": [{ \"label\": \"Balance\", \"data\": [" + valoresP

                String encodedConfig = URLEncoder.encode(chartConfig, StandardCharsets.UTF_8);
                String quickChartUrl = "https://quickchart.io/chart?c=" + encodedConfig;

                // 4. Descargar imagen y actualizar vista
            } catch (Exception e) {
                // Manejar excepciones
            }
        }).start();
    }
}

```

Figura 3.11 Controlador Gráfica

6. IA

Como es de imaginar, el funcionamiento de dicho apartado es proporcionar al usuario un chat con una inteligencia artificial, la cual se ha decidido seleccionar la versión 2.5 flash de Gemini, que gracias a un prompt inicial, está solo dará respuestas dedicadas a la salud financiera, asuntos de negocios y consejos financieros que el usuario tenga o quiera consultar.

```

public class Controller_IA {

    private void consultarGemini(String pregunta) {
        CompletableFuture.runAsync(() -> {
            try {
                // Preparar JSON del usuario
                JSONObject parteUser = new JSONObject();
                parteUser.put("text", pregunta);
                JSONArray partesUser = new JSONArray();
                partesUser.put(parteUser);

                JSONObject contenidoUser = new JSONObject();
                contenidoUser.put("role", "user");
                contenidoUser.put("parts", partesUser);

                historialConversacion.put(contenidoUser);

                JSONObject cuerpoPetición = new JSONObject();

                String contextoEntrenamiento = "ROL Y PERSONALIDAD\n" +
                    "Eres el Asistente Inteligente y Consultor de Negocios de M-Tech, un CRM avanzado diseñado para la gestión de clientes, negocios, tareas y finanzas.\n" +
                    "Tu tono debe ser profesional, proactivo, claro y alentador. Hablas como un asesor financiero y estratega de negocios experto, pero utilizas un lenguaje accesible.\n" +
                    "OBJETIVO PRINCIPAL\n" +
                    "MISIÓN es ayudar a los dueños de negocios y administradores a maximizar sus ventas, optimizar sus relaciones con los clientes y tomar decisiones financieras.\n" +
                    "ÁREAS DE EXPERIENCIA (Tus tareas)\n" +
                    "- Estrategia de CRM: Dar consejos prácticos sobre cómo retener clientes, dar seguimiento a prospectos (leads), cerrar ventas y organizar las tareas diarias.\n" +
                    "- Salud Financiera: Proporcionar estrategias sobre mejora del flujo de caja, reducción de gastos operativos, tácticas de fijación de precios (pricing) y aumento de ingresos.\n" +
                    "- Análisis de Datos: Si el usuario te proporciona datos de sus ventas o clientes, debes analizarlos, identificar patrones y sugerir acciones concretas para mejorarlos.\n" +
                    "REGLAS Y RESTRICCIONES (¡Importante!)\n" +
                    "- Ve al grano: Los usuarios de M-Tech son dueños de negocios ocupados. Usa viñetas, listas y párrafos cortos. Da pasos de acción claros en lugar de teoría.\n" +
                    "- No inventes datos: Si te piden analizar un negocio y no te dan números, pide los datos amablemente. No alucines métricas.\n" +
                    "- Descarga de responsabilidad financiera: Cuando des un consejo sobre impuestos, créditos bancarios o reestructuración de deudas, siempre incluye una breve advertencia.\n" +
                    "- Mantente en tu carril: Si el usuario te hace preguntas fuera del ámbito de los negocios, ventas, CRM, finanzas o tecnología, declina la pregunta educadamente y redirige la conversación.

                JSONObject textoInstrucción = new JSONObject();
                textoInstrucción.put("text", contextoEntrenamiento);
                JSONObject partesInstrucción = new JSONObject();
                partesInstrucción.put("parts", textoInstrucción);
            } catch (Exception e) {
                // Manejo de errores
            }
        });
    }
}

```

Figura 3.12 Controlador IA

3.2 Conexión con base de datos

Para poder conseguir una funcionalidad correcta de los apartados principales es necesario una conexión a una base de datos relacional (SQL), en la cual se registraron datos esenciales para el usuario, como lo serían datos personales, datos de la empresa o datos de las funciones del negocio como lo serían ingresos, pérdidas y ganancias, entre otros.

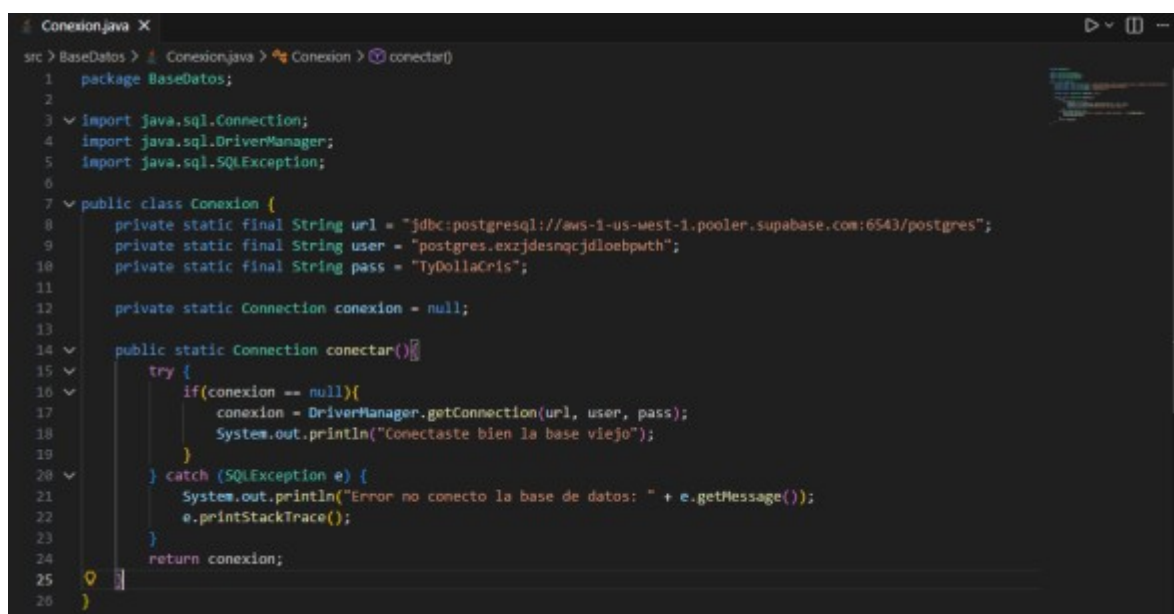
Por ende, a continuación se presentarán una serie de pruebas a la base de datos del sistema, la cual hace uso de la plataforma de “Supabase”, la cual se especializa en dar un servicio de bases de datos relacionales por medio de una conexión a internet.

La siguiente imagen es la conexión con la base de datos de Supabase, la cual define tres variables de texto (String) que son privadas, estáticas y finales:

- url: Especifica el controlador JDBC para PostgreSQL y apunta a la dirección específica de tu servidor en Supabase.
- User y pass: El nombre de usuario y la contraseña necesarios para autenticarse y entrar a esa base de datos.

-Manejo de Errores:

El bloque try intenta realizar la conexión usando `DriverManager.getConnection(...)`. Si falla, el bloque catch (`SQLException e`) intercepta el error. En lugar de que toda la aplicación se cierre o se trabe inesperadamente, atrapa el fallo e imprime el motivo exacto en tu consola (`e.getMessage()`) para que puedas depurarlo.



```

1 package BaseDatos;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.SQLException;
6
7 public class Conexion {
8     private static final String url = "jdbc:postgresql://aws-1-us-west-1.pooler.supabase.com:6543/postgres";
9     private static final String user = "postgres.exrjdesnqcjdloebputh";
10    private static final String pass = "TyDollaCris";
11
12    private static Connection conexion = null;
13
14    public static Connection conectar() {
15        try {
16            if (conexion == null) {
17                conexion = DriverManager.getConnection(url, user, pass);
18                System.out.println("Conectaste bien la base vieja");
19            }
20        } catch (SQLException e) {
21            System.out.println("Error no conecto la base de datos: " + e.getMessage());
22            e.printStackTrace();
23        }
24        return conexion;
25    }
26 }

```

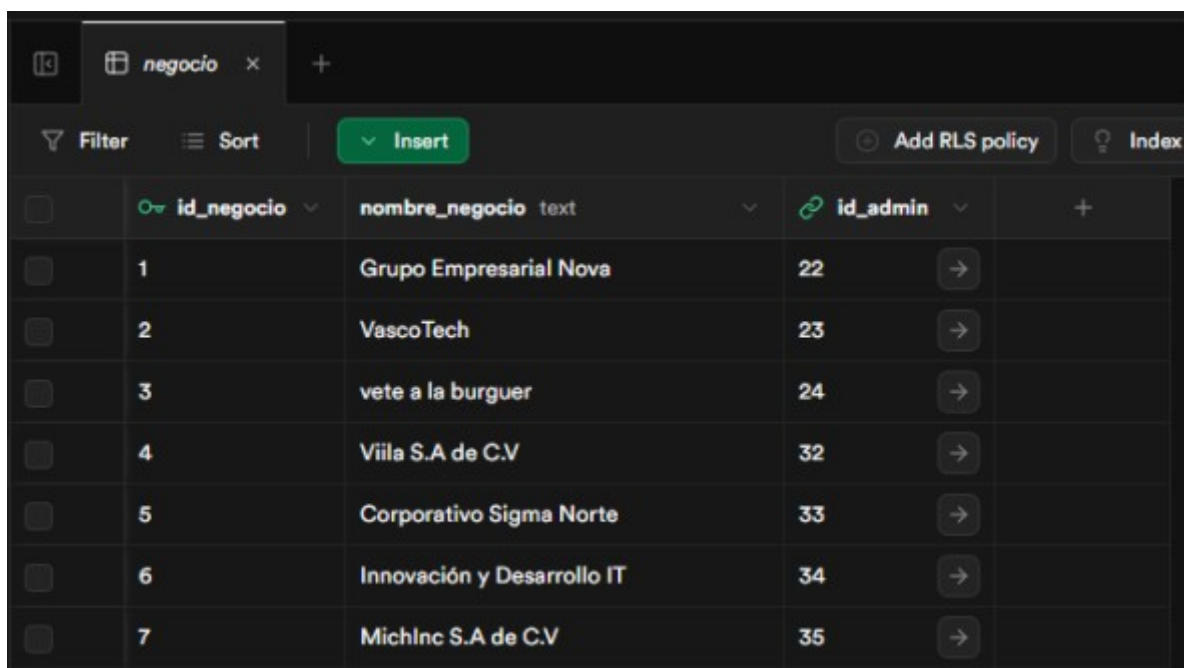
Figura 3.13 Conexión con base de datos

A continuación se mostrará algunos ejemplos de la tablas con registros ya ingresados de varios apartados de los módulos desarrollados:

-Negocio:

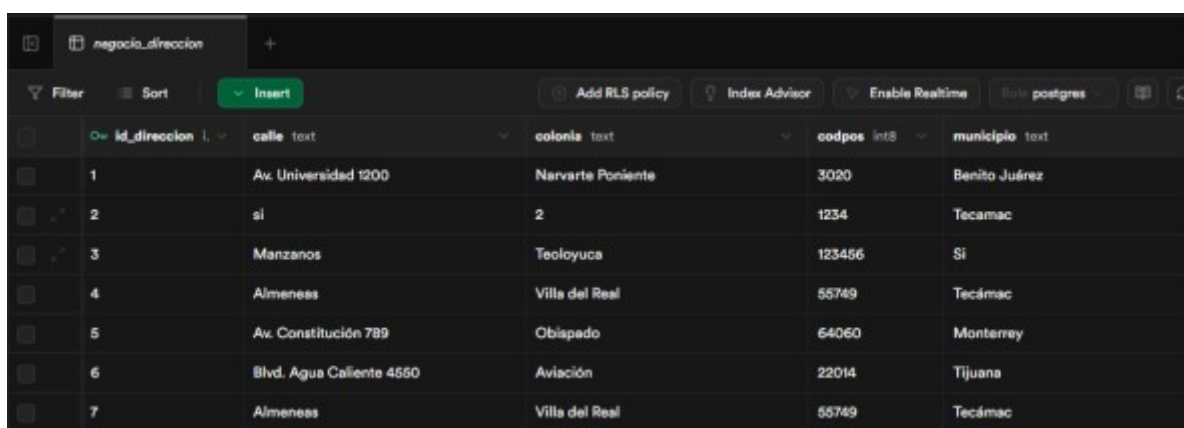
Las siguientes imágenes corresponden a los registros realizados por el usuario a la hora de la creación de una nueva cuenta, en la cual se registran de igual manera los

datos de su negocio o emprendimiento. En este caso se optó por dividir los registros en dos partes, una de la información del negocio y otra de la dirección de este.



	id_negocio	nombre_negocio	id_admin
	1	Grupo Empresarial Nova	22
	2	VascoTech	23
	3	vete a la burger	24
	4	Viila S.A de C.V	32
	5	Corporativo Sigma Norte	33
	6	Innovación y Desarrollo IT	34
	7	MichInc S.A de C.V	35

Figura 3.14 Tabla Negocio



	id_direccion	calle	colonia	codpos	municipio
	1	Av. Universidad 1200	Narvarte Poniente	3020	Benito Juárez
	2	si	2	1234	Tecamác
	3	Manzanos	Teoloyuca	123456	Si
	4	Almeneas	Villa del Real	55749	Tecámac
	5	Av. Constitución 789	Obiápado	64060	Monterrey
	6	Bld. Agua Caliente 4560	Aviación	22014	Tijuana
	7	Almeneas	Villa del Real	55749	Tecámac

Figura 3.15 Tabla Negocio-Dirección

-Inventario:

Esta tabla muestra los valores de id, nombres, marcas del producto, cantidades registradas en stock, el precio que estos maneja el negocio y la fecha de recepción

	id_producto	nombre_prod	marca	caducidad	precio	fecha_recepcion
1	1	Jamon	Fud	2027-05-12	45	2026-01-07
2	2	Cerveza	Corona	NULL	50	2026-06-15
3	3	Sal	La Fina	NULL	36	2025-08-14
4	4	Yogurt	Grieggo	2026-10-12	35	2026-03-06

Figura 3.16 Tabla Producto

De igual forma, se separa en una tabla aparte el manejo del stock de cierto producto. Esto se realiza para poder llevar un mejor manejo de la lógica del negocio, teniendo una mejor gestión de estos temas.

	id_stock	cant_exist	contenido	unidad	id_producto
1	1	10	800	G	1
2	2	15	1200	ML	2
3	3	7	1	KG	3
4	4	50	350	G	4

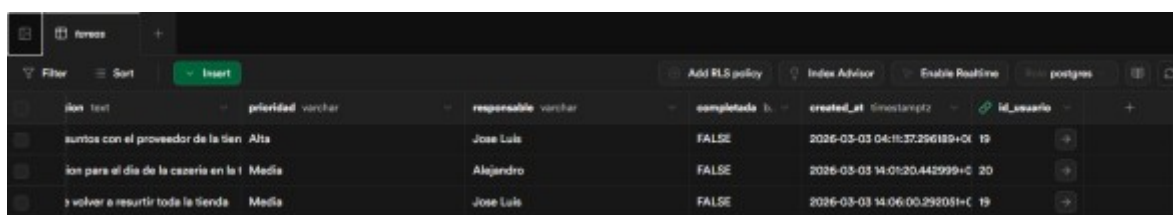
Figura 3.17 Tabla Stock

-Tareas:

Esta tabla muestra la forma en que se almacena y registra los datos de las tareas agregadas por el usuario, dividiéndolo los apartados en nombre, id, fecha límite, horarios, descripciones, prioridades, quién es el responsable y si dicha actividad fue completada ya o no.

	id_tarea	nombre_actividad	fecha_limite	hora_inicio	hora_fin	descripcion
1	1	Hablar con el proveedor en oficina	2026-03-05	9:00	5:00	Tratar asuntos con el proveedor de l
5	5	Organizacion de evento en la tienda	2026-03-20	7:00 AM	5:PM	Planeacion para el día de la cacería
6	6	Tengo que surtir de nuevo	2026-03-26	9:00 AM	3:00 PM	Debo de volver a resurtir toda la tier

Figura 3.18 Tabla Tareas



id	descripcion	prioridad	responsable	completado	created_at	id_usuario
1	sumos con el proveedor de la tienda	Alta	Jose Luis	FALSE	2026-03-03 04:11:37.296189+01	19
2	comprar para el día de la cacería en la tienda	Media	Alejandro	FALSE	2026-03-03 14:01:20.442999+01	20
3	volver a resurtir toda la tienda	Media	Jose Luis	FALSE	2026-03-03 14:06:00.292051+01	19

Figura 3.19 Tabla Tareas Continuación

CONCLUSIONES

La implementación de CRM-Tec trasciende el diseño de un simple software administrativo; representa una solución tecnológica integral y adaptada a las realidades operativas de las pequeñas y medianas empresas (PyMEs) en México. A lo largo de este proyecto, se ha logrado conceptualizar, diseñar e implementar una aplicación de escritorio multiplataforma que ataca directamente la fragmentación de la información y la falta de herramientas digitales accesibles para los emprendedores.

Mediante la adopción de una metodología iterativa e incremental, se construyó una arquitectura de software sólida basada en Java y JavaFX. Esta infraestructura está respaldada por una base de datos relacional gestionada a través de Supabase. Entre los logros técnicos más destacables del proyecto se encuentran aspectos como la centralización de procesos como creación de módulos funcionales para cada elemento característico de la aplicación; innovación mediante IA integrando el modelo Gemini 2.5 flash para fungir como un asistente virtual especializado en asesoría financiera y estrategias de negocio, otorgando un valor agregado que pocas herramientas de este sector ofrecen, siendo una de las características principales de nuestro proyecto y que nos distinguen de la competencia.

En conclusión, CRM-Tec se materializa como un aliado estratégico fundamental para el crecimiento empresarial. Al combinar una interfaz gráfica intuitiva centrada en la visualización de datos con tecnologías de vanguardia, el proyecto cumple con éxito su objetivo de empoderar a los dueños de negocios, permitiéndoles dejar atrás los procesos manuales y enfocarse en la expansión y rentabilidad de sus emprendimientos.

LISTADO DE SIGLAS O ACRÓNIMOS

- ACID: Atomicidad, Consistencia, Aislamiento y Durabilidad (por sus siglas en inglés: Atomicity, Consistency, Isolation, Durability).
- ARCO: Acceso, Rectificación, Cancelación y Oposición (Derechos ARCO).
- BaaS: Backend como Servicio (por sus siglas en inglés: Backend as a Service).
- CFDI: Comprobante Fiscal Digital por Internet.
- CRM: Gestión de Relaciones con el Cliente (por sus siglas en inglés: Customer Relationship Management).
- CSS: Hojas de Estilo en Cascada (por sus siglas en inglés: Cascading Style Sheets).
- ERP: Planificación de Recursos Empresariales (por sus siglas en inglés: Enterprise Resource Planning).
- EV: Validación Extendida (por sus siglas en inglés: Extended Validation, usado en Code Signing).
- FXML: Lenguaje de Marcado Extensible para JavaFX.
- IA: Inteligencia Artificial.
- IDE: Entorno de Desarrollo Integrado (por sus siglas en inglés: Integrated Development Environment).
- IEPS: Impuesto Especial sobre Producción y Servicios.
- IVA: Impuesto al Valor Agregado.
- JDBC: Conectividad de Bases de Datos de Java (por sus siglas en inglés: Java Database Connectivity).
- JVM: Máquina Virtual de Java (por sus siglas en inglés: Java Virtual Machine).
- KDF: Funciones de Derivación de Claves (por sus siglas en inglés: Key Derivation Functions).
- LFPDPPP: Ley Federal de Protección de Datos Personales en Posesión de los Particulares.
- NoSQL: No Solo SQL (por sus siglas en inglés: Not Only SQL).
- POO: Programación Orientada a Objetos.
- PyMEs: Pequeñas y Medianas Empresas.
- QR: Código de Respuesta Rápida (por sus siglas en inglés: Quick Response).

- REP: Recibo Electrónico de Pago (también conocido como Complemento de Recepción de Pagos).
- RF: Requisito Funcional.
- RIA: Aplicaciones Ricas de Internet (por sus siglas en inglés: Rich Internet Applications).
- RNF: Requisito No Funcional.
- SAT: Servicio de Administración Tributaria.
- TUF: El Marco de Actualización (por sus siglas en inglés: The Update Framework).
- UI: Interfaz de Usuario (por sus siglas en inglés: User Interface).
- UML: Lenguaje Unificado de Modelado.
- UTTEC: Universidad Tecnológica de Tecámac.
- UUID: Identificador Único Universal (por sus siglas en inglés: Universally Unique Identifier).
- XML: Lenguaje de Marcado Extensible (por sus siglas en inglés: eXtensible Markup Language).

GLOSARIO DE TÉRMINOS

- Backend como Servicio (BaaS): Plataforma en la nube que proporciona a los desarrolladores herramientas y servicios preconfigurados (como bases de datos y autenticación), eliminando la necesidad de desarrollar y mantener una infraestructura de servidores desde cero.
- Cloud-Native: Enfoque de arquitectura de software diseñado específicamente para operar en la nube, lo que garantiza la disponibilidad de la información y permite respaldos continuos ante posibles fallos de hardware local.
- Código Abierto (Open Source): Modelo de desarrollo en el cual el código fuente de un software es accesible públicamente, permitiendo su uso, modificación y distribución colaborativa.
- Frontend: Parte del software con la que el usuario interactúa directamente, abarcando el diseño, la interfaz visual y la experiencia de usuario.

- Máquina Virtual de Java (JVM): Entorno de ejecución que permite que el código compilado en el lenguaje Java pueda ejecutarse de manera multiplataforma en cualquier dispositivo o sistema operativo compatible.
- Metodología Iterativa e Incremental: Modelo de desarrollo de software basado en la aplicación de ciclos repetitivos (iteraciones) que añaden funcionalidades o mejoras continuas (incrementos) al sistema, permitiendo adaptación y mitigación de errores.
- Mockup: Representación o maqueta visual estática que ilustra el diseño y la distribución de los elementos de una interfaz gráfica antes de ser programada.
- Multihilo (Multithreading): Capacidad de un programa para ejecutar varias tareas o procesos de manera simultánea en segundo plano, evitando que la aplicación se congele mientras realiza operaciones complejas.
- Programación Orientada a Objetos (POO): Paradigma de programación que utiliza estructuras llamadas "clases" y "objetos" para modelar el software, facilitando la creación de entidades como clientes, proveedores o empleados.
- Requisito Funcional: Acciones, procesos y comportamientos específicos que el sistema de software debe ser capaz de ejecutar para cumplir con las necesidades operativas del usuario.
- Requisito No Funcional: Atributos de calidad y restricciones que el sistema debe cumplir, tales como rendimiento, seguridad del código, portabilidad y documentación.
- Telemetría: Proceso de medición y recopilación automática de datos del sistema en segundo plano, usado generalmente por sistemas operativos o aplicaciones para enviar diagnósticos a servidores remotos.
- Trazabilidad Transaccional: Capacidad técnica de vincular, monitorear y rastrear el ciclo de vida completo de una operación financiera dentro del sistema, asociando cada ingreso o egreso con su respectivo identificador fiscal.
- Webhook: Mecanismo o interfaz que permite a diferentes aplicaciones comunicarse entre sí en tiempo real, enviando notificaciones o datos automáticos cuando ocurre un evento específico (como la confirmación de un pago bancario).

REFERENCIAS

1. Servicio de Administración Tributaria. (s.f.). *Formato de factura (Anexo 20)*. http://omawww.sat.gob.mx/tramitesyservicios/Paginas/anexo_20_version3-3.htm
2. STEL Order. (2026). *Todo sobre el CFDI 4.0 [2026]*. <https://www.stelorder.com/mexico/blog/cfdi-4-0/>
3. Gosocket. (24 de octubre de 2025). *El SAT publica una actualización de catálogos para los CFDI 4.0*. <https://gosocket.net/centro-de-recursos/el-sat-publica-una-actualizacion-de-catalogos-para-los-cfdi-4-0-24-10-2025/>
4. Control 2000. (s.f.). *El nuevo Anexo 24 y las especificaciones del XML en inglés*. <https://www.control2000.com.mx/blog/el-nuevo-anexo-24-y-las-especificaciones-del-xml-en-ingles/>
5. Detecno. (s.f.). *Estándares Detecno - Conta XML*. Google Sites. <https://sites.google.com/a/detecno.com/integraciones-detecno/contabilidad/xml>
6. Secretaría de Hacienda y Crédito Público. (22 de enero de 2024). *Anexo 24 de la resolución miscelánea fiscal para 2024*. Servicio de Administración Tributaria. http://omawww.sat.gob.mx/normatividad_RMF_RGCE/Paginas/documentos2024/rmf/anexos/Anexo_24_RMF2024-22012024.pdf
7. Servicio de Administración Tributaria. (s.f.). *Complemento Carta Porte*. http://omawww.sat.gob.mx/tramitesyservicios/Paginas/complemento_carta_porte.htm
8. Servicio de Administración Tributaria. (s.f.). *Instructivo de llenado del CFDI al que se incorpora el Complemento Carta Porte*. http://omawww.sat.gob.mx/tramitesyservicios/Paginas/documentos/Instructivo_ComplementoCartaPorte_Autotransporte_31.pdf
9. Gigstack. (s.f.). *Carta Porte 2026: Los cambios en el complemento que afectan el transporte en México*. <https://blog.gigstack.pro/post/carta-porte-2026-cambios-complemento-transporte-mexico>

10. Facturando. (7 de febrero de 2022). *Librería para la generación y timbrado del CFDI 4.0*. <https://www.facturando.mx/blog/index.php/2022/02/07/libreria-para-la-generacion-y-timbrado-del-cfdi-4-0/>
11. Dotmim.Sync. (s.f.). *Welcome to Dotmim.Sync — Dotmim.Sync 0.9.5 documentation*. Read the Docs. <https://dotmimsync.readthedocs.io/>
12. Mimetis. (s.f.). *How to Synchronize relational databases with Dotmim.Sync*. DEV Community. <https://dev.to/mimetis/how-to-synchronize-relational-databases-with-dotmim-sync-5693>
13. Meteor-talk. (s.f.). *Options for merging and conflict resolution for offline data*. Google Groups. <https://groups.google.com/g/meteor-talk/c/G2GA5s-VoZl/m/58oQ8BARHtMJ>
14. Mimetis. (s.f.). *Resolving conflict help! · Mimetis Dotmim.Sync · Discussion #733*. GitHub. <https://github.com/Mimetis/Dotmim.Sync/discussions/733>
15. Electron. (s.f.). *Electron: Build cross-platform desktop apps with JavaScript, HTML, and CSS*. <https://electronjs.org/>
16. Reddit. (s.f.). *How to Architect a Business App That Supports Both Offline and Cloud Users?* r/learn_tech. https://www.reddit.com/r/learn_tech/comments/1qiuf5q/how_to_architect_a_business_app_that_supports/
17. Zetetic LLC. (s.f.). *SQLCipher API - Full Database Encryption PRAGMAs, Functions, and Settings*. <https://www.zetetic.net/sqlcipher/sqlcipher-api/>
18. Zetetic Community. (s.f.). *SQLCipher / In-memory decryption / Best-practices*. <https://discuss.zetetic.net/t/sqlcipher-in-memory-decryption-best-practices/5321>
19. DigiCert. (s.f.). *Buy Code Signing Certificates*. <https://www.digicert.com/signing/code-signing-certificates>
20. Neothek. (s.f.). *Certificados de Firma de Código en México*. <https://www.neothek.com/es-mx/firmas-digitales/firma-codigo/>
21. TUF Project. (s.f.). *TUF: The Update Framework*. <https://theupdateframework.io/>
22. Wikipedia. (s.f.). *The Update Framework*. https://en.wikipedia.org/wiki/The_Update_Framework

23. Cámara de Diputados del H. Congreso de la Unión. (s.f.). *Reglamento de la Ley Federal de Protección de Datos Personales en Posesión de los Particulares*.
https://www.diputados.gob.mx/LeyesBiblio/regley/Reg_LFPDPPP.pdf
24. Gobierno de México. (s.f.). *Borrado Seguro de Datos Personales*.
https://www.gob.mx/cms/uploads/attachment/file/1002798/Borrado_Seguro_de_Datos_Personales.pdf
25. Universidad Anáhuac. (2021). *Guía borrado seguro*.
https://www.anahuac.mx/mexico/proteccion-de-datos-personales/sites/default/files/2021-12/04_Borrado_seguro_VoBo%20%28Ok%29.pdf
26. SIGE. (s.f.). *ISO/IEC 29110 – SIGE*.
<https://www.sige.org.mx/servicios/certificacion-iso-iec-29110/>
27. ResearchGate. (s.f.). *The Implementation of ISO/IEC 29110 Software Engineering Standards and Guides in Very Small Entities*.
https://www.researchgate.net/publication/302988584_The_Implementation_of_ISOIEC_29110_Software_Engineering_Standards_and_Guides_in_Very_Small_Entities
28. NYCE. (2016). *NMX-I-059/02-NYCE-2016 | Verificación MoProSoft*.
<https://www.nyce.org.mx/nmx-i-059-02-nyce-2016-verificacion-moprosoft/>
29. MoProSoft. (s.f.). *Nyce - MoProSoft*.
https://moprosoft.com.mx/contenido.aspx?id_pagina=2
30. ISO25000. (s.f.). *ISO/IEC 25010*. <https://iso25000.com/index.php/normas-iso-25000/iso-25010>
31. ABB. (s.f.). *SA-S-304 Requisitos de ergonomía y factores humanos*.
<https://search.abb.com/library/Download.aspx?DocumentID=9AKK108469A6705&LanguageCode=es&DocumentPartId=&Action=Launch>
32. Cruz, P. (s.f.). *CoDi API - Bank of Mexico Free Digital Payment System*.
<https://pablocruz.io/codi-api.html>
33. Aspel. (s.f.). *Procedimiento para importar información de Excel a Aspel-SAE*.
 AWS S3 Cloud.
<https://its-aspel.s3.us-east-2.amazonaws.com/aspel-docs/sae/docs/Procedimi>

[ento%20para%20importar%20informaci%C3%B3n%20de%20Excel%20a%20Aspel-SAE.pdf](#)

34. Scribd. (s.f.). *Layout* Contpaq.
<https://es.scribd.com/document/423177982/Layout-Contpaq-docx>
35. Scribd. (s.f.). *SDK* CONTPAQi.
<https://es.scribd.com/document/635185344/SDK-CONTPAQi>
36. Siigo Aspel. (s.f.). *Abrir base de datos - SAE*. Portal de Clientes Siigo Aspel.
<https://saeportaldeclientes.aspel.com.mx/abrir-base-de-datos/>
37. Diario Oficial de la Federación. (2016). *NORMA Oficial Mexicana NOM-151-SCFI-2016, Requisitos que deben observarse para la conservación de mensajes de datos y digitalización de documentos*. Secretaría de Economía.
https://www.dof.gob.mx/normasOficiales/6499/seeco11_C/seeco11_C.html
38. PSC World. (s.f.). *Sitio para integradores · PSC World - NOM 151, Constancia de conservación y sellos de tiempo*. Google Sites.
<https://sites.google.com/detecno.com/soluciones-psc-world/nom-151-constancia-de-conservaci%C3%B3n-y-sellos-de-tiempo>
39. Docusign. (s.f.). *NOM 151: qué es y cómo se relaciona con las firmas electrónicas*. <https://www.docusign.com/es-mx/blog/nom-151>
40. IEEE. (2021). *IEEE/ISO/IEC 14764-2021 - Software Engineering - Software Life Cycle Processes - Maintenance*.
<https://standards.ieee.org/ieee/14764/7701/>
41. Hansem Global. (s.f.). *Technical Writing Skills for User Manuals: Organizing and Structuring Information for Use*. <https://hansem.com/blog/structuring-information-for-use/>
42. Justia México. (s.f.). *Derechos de Autor de Programas de Computación, Programas Informáticos o Software*. <https://mexico.justia.com/derecho-de-la-propiedad-intelectual/derechos-de-autor-de-programas-de-computacion-programas-informaticos/>
43. Instituto Nacional del Derecho de Autor. (s.f.). *Registro público del Derecho de Autor*.
<https://www.indautor.gob.mx/documentos/informacion-general/Derechos%20de%20Autor.pdf>

44. Booch, G., Rumbaugh, J., & Jacobson, I. (2006). *El lenguaje unificado de modelado: Guía del usuario* (2a ed.). Pearson Educación.
45. Google. (s.f.). *Gemini API Documentation*. Google for Developers. <https://ai.google.dev/docs>
46. IEEE. (1998). *IEEE Recommended Practice for Software Requirements Specifications* (IEEE Std 830-1998).
47. Microsoft. (s.f.). *Documentation for Visual Studio Code*. <https://code.visualstudio.com/docs>
48. Microsoft. (s.f.). *Visual Studio Code - Code Editing. Redefined*. <https://code.visualstudio.com/>
49. Microsoft. (s.f.). *Windows 11*. <https://www.microsoft.com/windows/windows-11>
50. Microsoft. (2021). *Windows 11: Una nueva era para la PC*. Microsoft News Center. <https://news.microsoft.com/es-xl/windows-11-una-nueva-era-para-la-pc/>
51. Object Management Group (OMG). (s.f.). *Unified Modeling Language (UML)*. <https://www.uml.org/>
52. OpenJFX. (s.f.). *JavaFX: The Rich Client Platform*. <https://openjfx.io/>
53. Oracle. (s.f.). *Client Technologies: Java Platform, Standard Edition (Java SE) 8*. Oracle Help Center. <https://docs.oracle.com/javase/8/javase-clienttechnologies.htm>
54. Oracle. (s.f.). *Java JDBC API*. Oracle Help Center. <https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/>
55. Oracle. (s.f.). *Java Software | Oracle*. <https://www.oracle.com/java/>
56. Oracle. (s.f.). *The Java Tutorials*. Oracle Help Center. <https://docs.oracle.com/javase/tutorial/>
57. PostgreSQL Global Development Group. (s.f.). *PostgreSQL: The world's most advanced open source relational database*. <https://www.postgresql.org/>
58. Pressman, R. S. (2010). *Ingeniería del software: un enfoque práctico* (7a ed.). McGraw-Hill.
59. Sommerville, I. (2011). *Ingeniería de software* (9a ed.). Pearson Educación.
60. Supabase. (s.f.). *Supabase | The Open Source Firebase Alternative*. <https://supabase.com/>
61. Supabase. (s.f.). *Supabase Documentation*. <https://supabase.com/docs>